

**INTEGRASI SISTEM AKSELERATOR CNN BERBASIS
KOMPUTASI FIXED-POINT 16-BIT PADA BOARD FPGA**

SKRIPSI



Disusun oleh:
ALEX CINATRA HUTASOIT
22/505820/TK/55377

**JURUSAN TEKNIK ELEKTRO DAN TEKNOLOGI INFORMASI
FAKULTAS TEKNIK UNIVERSITAS GADJAH MADA
YOGYAKARTA**

2026

HALAMAN PENGESAHAN

INTEGRASI SISTEM AKSELERATOR CNN BERBASIS KOMPUTASI FIXED-POINT 16-BIT PADA BOARD FPGA

SKRIPSI

Diajukan Sebagai Salah Satu Syarat untuk Memperoleh
Gelar Sarjana Teknik Program S-1
Pada Jurusan Teknik Elektro dan Teknologi Informasi Fakultas Teknik
Universitas Gadjah Mada

Disusun oleh:

Alex Cinatra Hutasoit
22/505820/TK/55377

Telah disetujui dan disahkan
pada tanggal 15 Juni 2026

Dosen Pembimbing I

Dosen Pembimbing II

Igi Ardiyanto, S.T., M.Eng.
NIP. 1987 0109 2020 12 1 005

Agus Bejo, S.T., M.Eng.
NIP. 1980 0101 2015 04 1 002

HALAMAN PERSEMBAHAN

*Untuk Ibu, Bapak,
dan Adik-adikku tercinta.*

KATA PENGANTAR

Assalamu'alaikum Wr. Wb.

Puji dan syukur ke hadirat Allah Yang Mahakuasa atas berkat dan karunia Roh Kudus sehingga tugas akhir ini dapat terselesaikan dengan baik. Penulis mengucapkan terima kasih atas arahan, bantuan, dukungan, dan doa yang telah diberikan oleh berbagai pihak. Pada kesempatan ini penulis mengucapkan terima kasih sedalam-dalamnya kepada pihak-pihak, di antaranya:

1. Bapak Prof. Ir. Hanung Adi Nugroho, S.T., M.Eng., Ph.D., IPM., SMIEEE., selaku Ketua Jurusan Teknik Elektro dan Teknologi Informasi Fakultas Teknik Universitas Gadjah Mada,
2. Bapak Dr.Eng. Ir. Igi Ardiyanto, S.T., M.Eng., IPM., ASEAN Eng., SMIEEE., selaku dosen pembimbing pertama yang telah memberikan banyak bantuan, bimbingan, serta arahan dalam Tugas Akhir ini,
3. Bapak Ir. Agus Bejo, S.T., M.Eng., D.Eng., IPM., selaku dosen pembimbing kedua yang juga telah memberikan banyak bantuan, bimbingan, serta arahan dalam Tugas Akhir dan kegiatan-kegiatan yang lain,
4. Bapak Widyan, S.T., M.Sc., Ph.D., selaku dosen pembimbing akademik yang telah memberikan pendampingan dan arahan selama masa perkuliahan,
5. Seluruh Dosen di Jurusan Teknik Elektro dan Teknologi Informasi FT UGM, yang tidak bisa disebutkan satu-satu, atas ilmu dan bimbingannya selama penulis berkuliah di JTETI,

Akhir kata penulis berharap semoga skripsi ini dapat memberikan manfaat dalam bidang teknologi informasi untuk penelitian selanjutnya di masa mendatang.

Wassalamu'alaikum Wr. Wb.

Yogyakarta, ... 2026

Alex Cinatra Hutasoit

DAFTAR ISI

HALAMAN PENGESAHAN	iii
HALAMAN PERSEMBAHAN	iii
KATA PENGANTAR	iv
DAFTAR ISI	viii
DAFTAR TABEL	ix
DAFTAR GAMBAR	x
DAFTAR SINGKATAN	xi
Intisari	xiv
<i>Abstract</i>	xv
I LATAR BELAKANG	1
1.1 Latar Belakang Masalah	1
1.2 Rumusan Masalah	2
1.3 Tujuan Penelitian	2
1.4 Batasan Masalah	2
1.5 Manfaat Penelitian	3
1.6 Sistematika Penulisan	3
II TINJAUAN PUSTAKA DAN DASAR TEORI	4
2.1 Tinjauan Pustaka	4
2.2 Landasan Teori	6
2.2.1 Backpressure	6
2.2.2 Finite State Machine	7
2.2.3 Multiply Accumulate	8
2.2.4 Time Multiplexer	9
2.2.5 Shift Register	9
2.2.6 First In First Out (FIFO)	11

2.2.7	Konvolusi	12
2.2.8	<i>Neural Network</i>	13
2.2.9	<i>Max Pool</i>	14
2.2.10	<i>Rectified Linear Unit</i> (ReLU)	15
2.2.11	CNN	16
2.2.12	Kuantitasi	17
2.2.13	Double Data Rate (DDR)	19
2.2.14	FPGA ARTIX 7	21
2.2.15	OV7670	22
2.2.16	VGA	22
2.3	Analisis Perbandingan Metode	23
III METODOLOGI PENELITIAN		25
3.1	Alat dan Bahan	25
3.1.1	Perangkat Keras	25
3.1.2	Perangkat Lunak	25
3.2	Rancangan Sistem	26
3.2.1	Rancangan Umum Sistem	26
3.2.2	Rancangan Arsitektur CNN	26
3.2.3	Rancangan Representasi Fixed-Point	27
3.2.4	Rancangan Akselerator CNN pada FPGA	28
3.2.5	Rancangan Sistem Kamera, DDR, dan VGA	32
3.3	Metode Implementasi	32
3.3.1	Implementasi Model CNN di Python	32
3.3.2	Implementasi Kuantisasi Fixed-Point	32
3.3.3	Implementasi CNN ke Verilog	32
3.3.4	Implementasi OV7670	32
3.3.5	Implementasi VGA	32
3.3.6	Implementasi DDR sebagai Frame Buffer	32
3.4	Metode Integrasi	32
3.4.1	Integrasi Model CNN ke FPGA	32
3.4.2	Integrasi OV7670 dengan DDR	32
3.4.3	Integrasi DDR dengan VGA	32
3.4.4	Integrasi OV7670, DDR, VGA, dan CNN	32
3.5	Metode Pengujian	32

3.5.1	Pengujian Akurasi Model di Python	32
3.5.2	Pengujian Perbedaan Hasil Python dan Verilog	32
3.5.3	Pengujian Penggunaan Kapasitas FPGA	32
3.5.4	Pengujian Sanitasi Kamera	32
3.5.5	Pengujian Sanitasi VGA	32
3.5.6	Pengujian DDR	33
3.5.7	Pengujian Sistem Terintegrasi	33
3.6	Metode Evaluasi	33
3.6.1	Evaluasi Akurasi	33
3.6.2	Evaluasi Perbedaan Bit	33
3.6.3	Evaluasi Latensi Inferensi	33
3.6.4	Evaluasi Penggunaan Resource FPGA	33
3.6.5	Evaluasi Daya	33
3.6.6	Evaluasi Tampilan Real Time	33
3.7	Tahapan Pelaksanaan	33
3.8	Jadwal Kegiatan	36
IV	HASIL DAN PEMBAHASAN	37
4.1	Hasil Implementasi Model CNN	37
4.1.1	Dataset yang Digunakan	37
4.1.2	Arsitektur Target Model CNN	38
4.1.3	Hasil Pelatihan Model CNN	38
4.1.4	Hasil Evaluasi Model CNN di Python	41
4.1.5	Kuantisasi Parameter Model CNN	43
4.2	Hasil Implementasi CNN Accelerator pada FPGA	43
4.2.1	Arsitektur CNN Accelerator pada FPGA	43
4.2.2	Hasil Penggunaan Resource CNN Accelerator	44
4.2.3	Hasil Timing CNN Accelerator	44
4.2.4	Hasil Latensi dan Throughput CNN Accelerator	44
4.3	Hasil Pengujian Modul Perangkat Keras	45
4.3.1	Hasil Uji Sanitasi OV7670	45
4.3.2	Hasil Uji Sanitasi VGA Pmod	45
4.3.3	Hasil Uji Akses DDR3 SDRAM	45
4.4	Hasil Integrasi Sistem	45
4.4.1	Hasil Integrasi OV7670, DDR, dan VGA	45

4.4.2	Hasil Integrasi CNN Accelerator dengan OV7670, DDR, dan VGA	45
4.4.3	Hasil Penggunaan Resource Sistem Terintegrasi	46
4.4.4	Hasil Timing Sistem Terintegrasi	46
4.4.5	Hasil Estimasi Daya Sistem Terintegrasi	46
4.5	Hasil Evaluasi dan Pembahasan	47
4.5.1	Evaluasi Perbedaan Hasil Python dan Verilog	47
4.5.2	Evaluasi Hasil Inferensi Model pada FPGA	49
4.5.3	Pembahasan Resource, Latensi, dan Daya	49
4.5.4	Pembahasan Ketercapaian Sistem terhadap Tujuan Penelitian .	49
V	KESIMPULAN DAN SARAN	50
5.1	Kesimpulan	50
5.2	Saran	50
	DAFTAR PUSTAKA	57
	LAMPIRAN	58
L.1	Lampiran Hasil <i>Running Synthetis CNN Accelerator</i>	58

DAFTAR TABEL

Tabel 2.1	Spesifikasi utama Arty A7-100T	21
Tabel 2.2	Perbandingan implementasi sistem pada penelitian terkait . .	23
Tabel 2.3	Perbandingan performa penelitian terkait	24
Tabel 3.1	Target penggunaan resource pada implementasi CNN	29
Tabel 3.2	Jadwal Penelitian.	36
Tabel 4.1	Ukuran Parameter Pada Model Python	40
Tabel 4.2	Parameter Model pada Python	42
Tabel 4.3	Hasil Kuantisasi Parameter	43
Tabel 4.4	Hasil <i>CNN Implement Running</i>	44
Tabel 4.5	Hasil Routing CNN Accelerator Design	44
Tabel 4.6	Hasil <i>CNN Accelerator Delayed Design</i>	44
Tabel 4.7	Hasil Latensi CNN Accelerator	44
Tabel 4.8	Hasil <i>Streaming Flow Implement Running</i>	45
Tabel 4.9	Detail Hasil <i>Implement Running</i>	46
Tabel 4.10	Hasil Routing Design	46
Tabel 4.11	Hasil <i>Delayed Design</i>	46
Tabel 4.12	Penggunaan Daya dan Temperature	46

DAFTAR GAMBAR

Gambar 2.1	Aliran sinyal <i>valid</i> dan <i>ready</i> pada sistem pipeline	6
Gambar 2.2	Ilustrasi data <i>input shift register</i>	10
Gambar 2.3	Ilustrasi FIFO	11
Gambar 2.4	Operasi Konvolusi	13
Gambar 2.5	Operasi Max Pool	15
Gambar 2.6	Ilustrasi hubungan AXI, MIG, dan DDR pada sistem pemindaian data berbasis memori eksternal.	19
Gambar 3.1	Rancangan Umum Sistem	26
Gambar 3.2	Diagram Alir Penelitian	35
Gambar 4.1	Distribusi <i>Training Dataset</i> yang digunakan	37
Gambar 4.2	Arsitektur Model CNN	38
Gambar 4.3	<i>Loss training</i> model	38
Gambar 4.4	<i>Accuracy training</i> model	39
Gambar 4.5	<i>Confusion Matrix Training</i> model	40
Gambar 4.6	Distribusi <i>Test Dataset</i> yang digunakan	41
Gambar 4.7	<i>Confusion Matrix testing</i> model	42
Gambar 4.8	Desain CNN Accelerator pada FPGA	43
Gambar 4.9	Desian Akhir Integrasi	45
Gambar 4.10	Detail Hasil Penggunaan Daya	47
Gambar 4.11	Perbandingan <i>Bit</i> dan <i>Float value</i> pada <i>Conv 1</i> dan <i>Pool 1</i> . .	47
Gambar 4.12	Perbandingan <i>Bit</i> dan <i>Float value</i> pada <i>Conv 2</i> dan <i>Pool 2</i> . .	48
Gambar 4.13	Perbandingan <i>Bit</i> dan <i>Float value</i> pada <i>Conv 3</i> dan <i>Pool 3</i> . .	48
Gambar 4.14	Perbandingan <i>Bit</i> dan <i>Float value</i> pada <i>Dense 1</i> dan <i>Dense 2</i> .	48
Gambar 4.15	Perbedaan Akurasi	49

DAFTAR SINGKATAN

A

AXI Advanced eXtensible Interface

B

BD Block Design

BRAM Block Random Access Memory

BUFG Global Clock Buffer

C

CLK Clock

CNN Convolutional Neural Network

CONV Convolution

CPU Central Processing Unit

CSV Comma-Separated Values

D

DRC Design Rule Check

DSP Digital Signal Processing

DDR Double Data Rate

F

FF Flip Flop

FIFO First In First Out

FWFT First Word Fall Through

FPGA Field Programmable Gate Array

FPS Frame Per Second

G

GPIO General-Purpose Input/Output

GPU Graphics Processing Unit

H

HDL Hardware Descriptive Language

I

I/O Input Output

I2C Inter-Integrated Circuit

IOB Input/Output Block

J

JSON Javascript Object Notation

L

LUT Look-Up Table

M

MIG Memory Interface Generator

MNIST Modified National Institute of Standards and Technology

MUX Multiplexer

N

NPZ NumPy Compressed Archive

S

SCCB Serial Camera Control Bus

SOC System On Chip

SPI Serial Peripheral Interface

T

TNS Total Negative Slack

THS Total Hold Slack

U

UART Universal Asynchronous Receiver-Transmitter

UI User Interface

URAM Ultra Random Access Memory

W

WNS Worst Negative Slack

WHS Worst Hold Slack

WPWS Worst Pulse Width Slack

Intisari

....

Kata kunci : *CNN*, *ARTY 7*, *OV7670*, Serial, Komputasi Pararel.

Abstract

....

Keywords : *CNN, ARTY 7, OV7670*, Serial, Pararel Computation.

BAB I

LATAR BELAKANG

1.1 Latar Belakang Masalah

Kemampuan CNN dalam melakukan ekstraksi fitur hirarkis pada spatial data matriks menjadikannya sebagai salah satu metode yang paling banyak digunakan dalam proses pengolahan citra dan visi. Proses ekstraksi fitur pada data gambar berskala besar dilakukan oleh secara otomatis dan menjadikan CNN sebagai fondasi utama dalam implementasi pengolahan citra dan visi untuk pengenalan pola, objek hingga karakter [1, 2]. Keunggulan ini mendorong adopsi dan implementasi di berbagai bidang terutama sistem tertanam. Saat ini implementasi CNN banyak dilakukan di GPU dan CPU. Konsumsi daya pada GPU sangat besar dan boros baik pada saat proses pelatihan model maupun pada saat mode inferensial[3, 4]. Hal ini terjadi terutama pada GPU, karena overhead komunikasi data bus pada batch berukuran kecil sehingga penggunaan daya sangat tidak efisien.

Keterbatasan implementasi CNN pada CPU dan GPU menjadi semakin penting ketika CNN diterapkan pada sistem tertanam yang membutuhkan pemrosesan citra secara langsung, cepat, dan hemat daya[5]. Pada aplikasi seperti kamera cerdas, robotika, dan kendaraan otonom, data visual diperoleh secara terus-menerus dari sensor sehingga proses komputasi tidak cukup hanya akurat, tetapi juga harus memiliki latensi rendah. Oleh karena itu, pendekatan *edge computation* dan *embedded vision* menjadi salah satu solusi karena proses inferensi dapat dilakukan lebih dekat dengan sumber data.

Salah satu alternatif perangkat keras yang dapat digunakan untuk mempercepat komputasi CNN adalah FPGA. FPGA memiliki keunggulan karena arsitektur perangkat kerasnya dapat dikonfigurasi sesuai kebutuhan algoritma. Berbeda dengan CPU dan GPU yang menggunakan arsitektur komputasi umum, FPGA memungkinkan perancang untuk membangun jalur data khusus, menerapkan paralelisme pada tingkat operasi, serta menggunakan teknik *pipeline* untuk meningkatkan *throughput*. Karakteristik ini sesuai dengan operasi CNN yang bersifat berulang, terstruktur, dan memiliki pola komputasi yang dapat dipetakan ke dalam perangkat keras pola [6, 7].

Penelitian ini berfokus pada perancangan dan implementasi akselerator CNN berbasis FPGA dengan komputasi *fixed-point* yang berjalan dengan daya rendah. Ak-

selerator dirancang untuk menjalankan proses inferensi CNN secara perangkat keras dengan memanfaatkan paralelisme, *pipeline*, dan optimasi representasi data. Selain itu, penelitian ini juga membahas integrasi sistem dengan perangkat kamera dan jalur tampilan citra, sehingga sistem tidak hanya diuji sebagai modul komputasi terpisah, tetapi juga sebagai bagian dari sistem *embedded vision* yang menerima data visual dari lingkungan nyata.

1.2 Rumusan Masalah

Implementasi CNN pada CPU dan GPU masih membutuhkan *resource* komputasi yang besar, terutama dari sisi konsumsi daya dan efisiensi inferensi pada sistem tertanam. Upaya untuk menekan konsumsi daya pada perangkat komputasi umum sering kali berdampak pada penurunan performa sehingga latensi inferensi menjadi lebih besar. Sedangkan kebutuhan akan sistem yang efisien secara daya dan instan semakin dibutuhkan.

1.3 Tujuan Penelitian

Tujuan penelitian ini adalah untuk menunjukkan potensi FPGA sebagai platform alternatif di samping CPU dan GPU dalam implementasi CNN hingga tahap *deployment* pada sistem perangkat keras.

1.4 Batasan Masalah

Batasan masalah pada penelitian ini adalah:

1. Pada penelitian ini *board* FPGA yang digunakan adalah ARTY 7 100T.
2. Kamera yang digunakan adalah OV7670.
3. Streaming video dilakukan lewat VGA, dengan modul VGA PMOD Digilent.
4. Keterbatasan *resource* pada FPGA memungkinkan penelitian menggunakan model yang tidak terlalu besar.
5. *Dataset* yang digunakan adalah *Dataset* MNIST, dengan ukuran 28x28 *pixel channel* 1.
6. *Training* model dilakukan di CPU python.

1.5 Manfaat Penelitian

Penelitian ini diharapkan dapat memberikan kontribusi dalam pengembangan implementasi CNN pada perangkat FPGA, khususnya pada penggunaan komputasi *fixed-point*, pemetaan paralelisme operasi konvolusi, dan penerapan *pipeline* untuk mempercepat proses inferensi. Selain itu, penelitian ini dapat menjadi referensi dalam memahami proses perancangan akselerator CNN berbasis perangkat keras untuk aplikasi *embedded vision*.

1.6 Sistematika Penulisan

BAB I : PENDAHULUAN

Pada bab ini dijelaskan latar belakang, rumusan masalah, batasan, tujuan, manfaat, dan sistematika penulisan.

BAB II : TINJAUAN PUSTAKA DAN LANDASAN TEORI

Pada bab ini dijelaskan teori-teori dan penelitian terdahulu yang digunakan sebagai acuan dan dasar dalam penelitian.

BAB III : METODOLOGI PENELITIAN

Pada bab ini dijelaskan metode yang digunakan dalam penelitian meliputi langkah kerja, pertanyaan penelitian, alat dan bahan, serta tahapan dan alur penelitian.

BAB IV : HASIL DAN PEMBAHASAN

Pada bab ini dijelaskan hasil penelitian dan pembahasannya.

BAB V : KESIMPULAN DAN SARAN

Pada bab ini ditulis kesimpulan akhir dari penelitian dan saran untuk pengembangan penelitian selanjutnya.

BAB II

TINJAUAN PUSTAKA DAN DASAR TEORI

2.1 Tinjauan Pustaka

Penelitian Che et al. dan Qasaimeh et al. menunjukkan bahwa pemilihan platform komputasi untuk aplikasi visi sangat dipengaruhi oleh karakteristik algoritma, latensi, dan kebutuhan energi [8, 9]. CPU memiliki fleksibilitas tinggi, tetapi kurang efisien untuk komputasi CNN yang membutuhkan paralelisme besar. GPU mampu memberikan throughput tinggi, namun umumnya membutuhkan daya dan bandwidth memori yang lebih besar. Dalam konteks tersebut, FPGA menjadi alternatif karena operasi CNN seperti konvolusi, *multiply-accumulate*, *buffering*, dan *pipeline* dapat dipetakan langsung ke sumber daya perangkat keras seperti DSP, BRAM, LUT, dan FF, sehingga sesuai untuk sistem *embedded vision* yang membutuhkan latensi rendah dan konsumsi daya efisien [10].

Berdasarkan penelitian Solovyev et al., implementasi CNN pada FPGA untuk *real-time processing* dilakukan dengan mengubah representasi *floating-point* menjadi *fixed-point* agar lebih sesuai dengan komputasi perangkat keras [11]. Penelitian ini menggunakan dataset MNIST dengan augmentasi berupa rotasi acak 10 derajat, inversi warna, serta penambahan *noise* dari 0% sampai 10%. Setelah proses pelatihan, parameter CNN disimpan ke dalam blok memori FPGA dan digunakan pada proses inferensi. Implementasi dilakukan pada board *DE0-Nano* berbasis FPGA *Intel Cyclone IV*, dengan masukan citra dari kamera OV7670 dan keluaran tampilan melalui VGA. Citra dari kamera direduksi menjadi *grayscale* berukuran 28×28 piksel untuk diproses oleh CNN, sehingga sistem mampu berjalan secara *real-time* dengan kecepatan lebih dari 150 *frame per second*. Penelitian ini menghasilkan akurasi model yang tidak berubah dengan model training sebelum kuantisasi.

Pada sisi kamera, penelitian Solovyev et al. juga memperlihatkan penggunaan OV7670 sebagai sumber citra untuk sistem CNN berbasis FPGA [11]. Konfigurasi kamera OV7670 dilakukan melalui *Serial Camera Control Bus* atau SCCB untuk mengatur format keluaran citra, resolusi, dan mode operasi sensor [12]. Data piksel kemudian dikirim melalui *pixel clock*, bus data paralel, serta sinyal sinkronisasi *HREF* dan *VSYNC*. Sinyal *HREF* menandai data piksel valid pada satu baris, sedangkan *VSYNC* menandai pergantian *frame*. Sistem inferensi *end-to-end* CNN

berbasis kamera tidak hanya membutuhkan akselerator inferensi, tetapi juga akuisisi citra yang sinkron terhadap tampilan VGA.

Penelitian Qiu et al. mengimplementasikan CNN pada FPGA *Xilinx ZC706* dengan model *VGG16-SVD* untuk klasifikasi citra ImageNet menggunakan representasi 16-bit *fixed-point* [13]. Hasil implementasi menunjukkan akurasi *top-5* sebesar 86.66%, performa inferensi sebesar 4.45 FPS, *throughput* sekitar 137 GOPS untuk keseluruhan jaringan, konsumsi daya sebesar 9.63 W. Penelitian ini menunjukkan bahwa kuantisasi 16-bit *fixed-point* masih dapat mempertahankan akurasi yang layak pada model CNN berskala besar. Dari sisi arsitektur, operasi konvolusi menjadi bagian komputasi dominan sehingga pemetaan operasi *multiply-accumulate* ke DSP, penggunaan *buffer*, serta pengaturan akses memori menjadi faktor penting dalam meningkatkan *throughput*.

Penelitian Dua et al. melalui arsitektur *Systolic-CNN* menunjukkan implementasi CNN berbasis *OpenCL* pada FPGA dengan pendekatan yang lebih fleksibel terhadap banyak model [14]. Berbeda dari Solovyev et al. dan Qiu et al. yang menggunakan *fixed-point*, penelitian ini menggunakan format 32-bit *floating-point*. Model yang diuji meliputi *AlexNet*, *ResNet-50*, *ResNet-152*, *RetinaNet*, dan *Light-weight RetinaNet*. Implementasi dilakukan pada board *Intel Arria 10 GX1150* dan *Intel Stratix 10 GX2800*. Pada *Arria 10*, inferensi *AlexNet* membutuhkan waktu sekitar 7 ms dan *ResNet-50* sekitar 84 ms, sedangkan pada *Stratix 10* latensinya menjadi sekitar 2 ms dan 33 ms. Paper ini tidak melaporkan konsumsi daya maupun efisiensi energi dalam GOPS/W, sehingga lebih tepat digunakan sebagai referensi arsitektur *systolic array*, pemanfaatan DSP, dan fleksibilitas *OpenCL*.

Selain penelitian CNN, integrasi kamera dan tampilan juga ditunjukkan oleh Navaneethan et al. melalui implementasi *video streaming* dari kamera OV7670 ke VGA pada board FPGA *Spartan-6* dan *Artix-7* menggunakan Verilog dan Xilinx Vivado [15]. Penelitian ini tidak berfokus pada inferensi CNN, tetapi menunjukkan bahwa jalur akuisisi kamera, sinkronisasi piksel, dan tampilan VGA dapat diimplementasikan dengan penggunaan sumber daya yang relatif kecil, yaitu 530 *slice registers*. Hasil tersebut relevan sebagai pembandingan bagian antarmuka kamera dan display, terutama karena sistem CNN berbasis kamera tetap membutuhkan jalur video yang stabil sebelum citra diproses oleh akselerator.

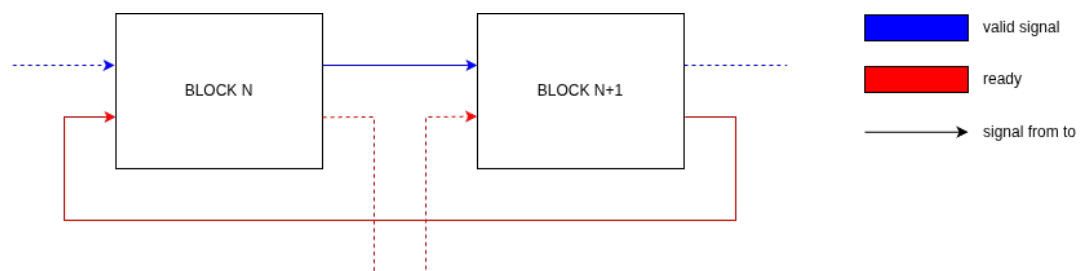
Pada sistem yang memproses citra berukuran besar, penggunaan *frame buffer* menjadi penting karena data kamera dan data tampilan bekerja pada aliran waktu yang berbeda. Untuk resolusi VGA 640×480 dengan format RGB, kapasitas BRAM

pada FPGA umumnya tidak cukup untuk menyimpan satu atau dua *frame* penuh [16]. Oleh karena itu, pada rancangan ini DDR digunakan sebagai *frame buffer*, sedangkan akses baca dan tulis dikendalikan melalui antarmuka AXI menuju MIG. AXI bertindak sebagai jalur transaksi antara modul *master* dan MIG sebagai pengendali DDR, dan proses akses memori dilakukan setelah MIG selesai dikalibrasi [17, 18].

2.2 Landasan Teori

2.2.1 Backpressure

Backpressure adalah mekanisme pengendalian aliran data yang digunakan ketika suatu blok belum siap menerima data dari blok sebelumnya. Mekanisme ini umumnya bekerja dengan sinyal *valid* dan *ready*. Sinyal *valid* menunjukkan bahwa data dari pengirim sudah tersedia, sedangkan sinyal *ready* menunjukkan bahwa penerima siap mengambil data tersebut [19, 20].



Gambar 2.1: Aliran sinyal *valid* dan *ready* pada sistem pipeline

Berdasarkan Gambar 2.1, sinyal *valid* mengalir searah dengan aliran data, yaitu dari blok proses sebelumnya menuju blok proses berikutnya. Sebagai contoh, ketika blok proses ke- N menghasilkan data, blok tersebut akan mengaktifkan sinyal *valid* ke blok proses ke- $N+1$. Sebaliknya, sinyal *ready* mengalir ke arah belakang, yaitu dari blok penerima menuju blok pengirim. Sinyal ini digunakan untuk memberitahu bahwa blok penerima sudah siap menerima dan memproses data berikutnya.

Jika sinyal *ready* belum aktif, maka pengirim harus menahan data sampai penerima siap. Dengan cara ini, data tidak hilang atau tertimpa oleh data berikutnya. Pada sistem berbasis *pipeline*, konsep backpressure penting untuk menjaga aliran data tetap teratur, terutama ketika setiap blok memiliki waktu proses yang berbeda [19].

2.2.2 Finite State Machine

Finite State Machine atau FSM merupakan model kendali sekuensial yang merepresentasikan perilaku sistem dalam sejumlah *state* terbatas. Pada setiap siklus kerja, FSM membaca *input*, mengevaluasi kondisi transisi, menentukan *state* berikutnya, kemudian menghasilkan *output* sesuai dengan aturan transisi yang telah ditentukan seperti pada *Pseudocode* 1. Dengan pendekatan ini, proses kerja yang kompleks dapat dibagi menjadi beberapa tahap yang lebih terstruktur, sehingga alur kendali sistem menjadi lebih mudah dirancang, diamati, dan diverifikasi.

Algorithm 1 Finite State Machine Algorithm

Require: inputSignal, currentState, TransitionSet

Ensure: updated currentState, outputSignal

```

1: procedure UPDATEFSM(inputSignal)                                ▷ Initialize default values
2:   nextState ← currentState
3:   outputSignal ← DEFAULT_OUTPUT
                                     ▷ Evaluate all possible state transitions
4:   for all transition ∈ TransitionSet do
5:     if transition.sourceState == currentState and transition.inputCondition(inputSignal) == true then
                                     ▷ State transition
6:       nextState ← transition.targetState
                                     ▷ Generate output
7:       outputSignal ← transition.outputAction(currentState, inputSignal,
nextState)
8:       break
9:     end if
10:  end for
                                     ▷ Update the current state
11:  currentState ← nextState
                                     ▷ Return the generated output
12:  return outputSignal
13: end procedure

```

Dalam sistem digital, FSM banyak digunakan untuk mengatur proses yang membutuhkan urutan kerja tertentu, seperti proses inisialisasi, pembacaan data, penulisan data, sinkronisasi sinyal, dan pengendalian jalur *data path*. Pada penelitian ini, FSM digunakan sebagai bagian penting dalam pengendalian proses *input-output*, kendali transaksi AXI, kendali kamera, serta pengaturan tahapan komputasi pada CNN *Accelerator*. Kebutuhan kendali tersebut menjadi semakin penting pada implementasi

CNN di FPGA, karena proses komputasi CNN terdiri dari beberapa operasi berurutan seperti konvolusi, aktivasi, *pooling*, dan klasifikasi. CNN sendiri banyak digunakan pada tugas klasifikasi citra karena kemampuannya dalam mengekstraksi fitur secara bertingkat [2, 1]. Oleh karena itu, FSM berperan sebagai pengatur aliran proses agar setiap modul dapat bekerja secara sinkron, mulai dari penerimaan data citra, pemrosesan fitur, hingga keluaran hasil klasifikasi.

Pada sistem yang lebih kompleks, transisi FSM tidak hanya ditentukan oleh *input* utama, tetapi juga oleh kondisi kesiapan antarblok. Mekanisme ini berkaitan dengan *backpressure*, yaitu kondisi ketika sebuah modul harus menahan proses karena modul berikutnya belum siap menerima data. Dalam implementasi *advanced backpressure*, FSM dapat diperluas dengan sinyal kendali seperti *valid*, *ready*, *busy*, *done*, *fifo empty*, dan *fifo full*. Sinyal-sinyal tersebut digunakan sebagai syarat transisi agar perpindahan *state* hanya terjadi ketika data tersedia dan jalur keluaran siap menerima data. Dengan demikian, FSM tidak hanya berfungsi sebagai pengatur urutan kerja, tetapi juga sebagai mekanisme kontrol aliran data untuk mencegah kehilangan data, pembacaan data kosong, atau penulisan data ketika buffer sudah penuh.

2.2.3 Multiply Accumulate

Multiply Accumulate (MAC) adalah operasi komputasi yang terdiri dari proses perkalian dua nilai dan penjumlahan hasilnya ke nilai akumulasi sebelumnya. Secara umum, operasi MAC dapat dituliskan sebagai berikut:

$$y = \sum_{i=0}^{N-1} x_i w_i + b \quad (2.1)$$

Pada persamaan 2.1, x_i merupakan data masukan, w_i merupakan bobot, dan b merupakan nilai bias. Operasi ini banyak digunakan pada komputasi (CNN), terutama pada proses konvolusi dan *fully connected layer*. Pada proses tersebut, setiap nilai keluaran diperoleh dari hasil perkalian antara data masukan dan bobot, kemudian seluruh hasil perkalian tersebut dijumlahkan.

Pada FPGA, operasi MAC dapat diimplementasikan menggunakan blok logika umum atau menggunakan sumber daya khusus berupa DSP. Pada FPGA seri 7, blok DSP48E1 memiliki struktur yang mendukung operasi perkalian, penjumlahan, dan akumulasi, sehingga sesuai digunakan untuk mempercepat operasi MAC [21]. Penggunaan DSP membuat operasi MAC lebih efisien dibandingkan jika seluruh ope-

rasi aritmetika dibangun hanya menggunakan LUT dan FF.

Namun, jumlah DSP pada FPGA bersifat terbatas. Jika setiap operasi perkalian dan akumulasi pada CNN dibuat secara paralel penuh, maka kebutuhan DSP dapat meningkat sangat besar [22]. Oleh karena itu, perancangan akselerator CNN pada FPGA perlu mempertimbangkan keseimbangan antara jumlah operasi paralel, penggunaan DSP, dan waktu komputasi.

2.2.4 Time Multiplexer

Time multiplexer adalah teknik penggunaan satu sumber daya hardware secara bergantian pada waktu yang berbeda. Dalam akselerator CNN, teknik ini digunakan ketika jumlah operasi MAC yang dibutuhkan lebih banyak dibandingkan jumlah DSP yang tersedia. Dengan teknik ini, satu unit MAC tidak hanya digunakan untuk satu operasi saja, tetapi dapat dipakai berulang untuk memproses beberapa data, channel, filter, atau neuron secara berurutan.

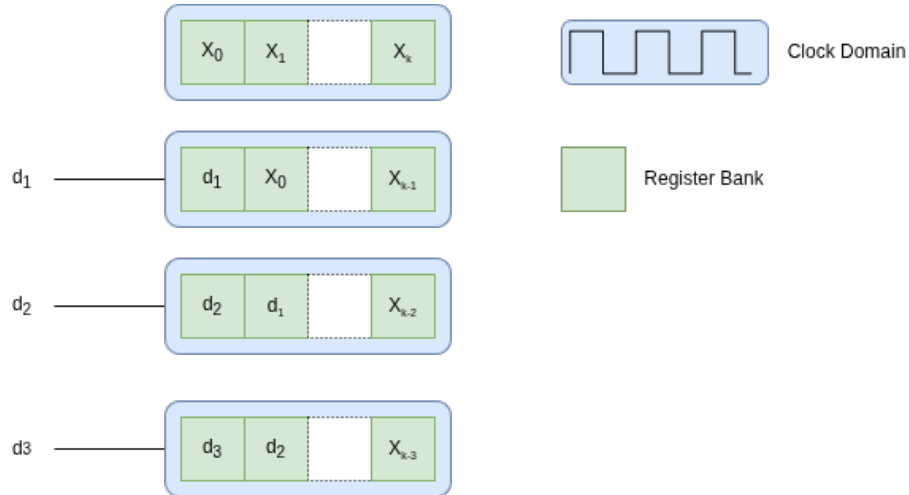
Berbeda dengan paralel penuh yang menyediakan banyak unit MAC sekaligus, time multiplexer menggunakan pendekatan berbasis pembagian waktu. Setiap siklus clock atau beberapa siklus clock digunakan untuk memproses bagian data tertentu, kemudian unit komputasi yang sama digunakan kembali untuk bagian data berikutnya [23, 14, 24]. Pendekatan ini dapat mengurangi penggunaan resource FPGA, tetapi waktu komputasi dapat menjadi lebih panjang karena sebagian operasi dilakukan secara bergantian.

Time multiplexing berkaitan erat dengan keterbatasan resource seperti DSP, LUT, dan memori internal [25]. Beberapa penelitian akselerator CNN pada FPGA menggunakan pendekatan *resource multiplexing* untuk mengurangi konsumsi hardware dengan cara memakai kembali unit komputasi yang sama pada beberapa tahap proses [22]. Pendekatan serupa juga digunakan pada arsitektur akselerator neural network yang menyesuaikan jumlah resource komputasi terhadap kebutuhan throughput tiap layer [24].

2.2.5 Shift Register

Shift register adalah rangkaian register yang digunakan untuk menyimpan dan menggeser data dari satu posisi ke posisi berikutnya pada setiap siklus *clock*. Pada sistem digital, shift register banyak digunakan ketika data harus diproses secara berurutan, terutama pada aliran data yang masuk secara *stream*. Setiap data baru yang

masuk akan mendorong data sebelumnya ke posisi berikutnya, sehingga beberapa data terakhir dapat tetap tersedia secara bersamaan di dalam rangkaian register.



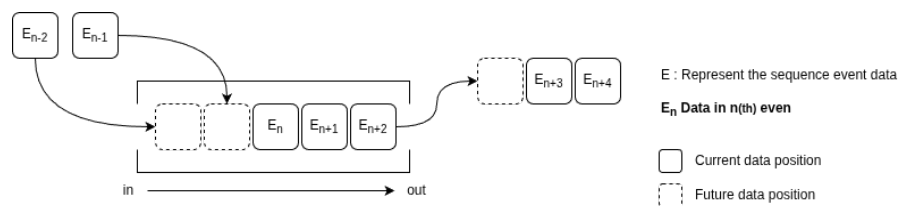
Gambar 2.2: Ilustrasi data *input shift register*.

Pada operasi konvolusi citra, shift register memiliki peran penting karena kernel tidak hanya membutuhkan satu piksel, tetapi juga piksel-piksel tetangga di sekitarnya. Misalnya pada kernel berukuran 3×3 , proses konvolusi membutuhkan sembilan nilai piksel yang berasal dari tiga baris dan tiga kolom yang berdekatan [2]. Jika data citra masuk secara berurutan dari kiri ke kanan dan dari atas ke bawah, maka sistem tidak dapat langsung memperoleh sembilan piksel tersebut tanpa mekanisme penyimpanan sementara. Oleh karena itu, shift register digunakan untuk mempertahankan beberapa piksel terakhir dalam satu baris, sehingga susunan piksel horizontal dapat dibentuk secara bertahap.

Dalam akselerator CNN berbasis FPGA, shift register membantu membentuk *window* konvolusi secara langsung dari aliran data. Setelah data piksel masuk ke dalam jalur pemrosesan, nilai-nilai piksel tersebut digeser pada setiap siklus *clock* hingga membentuk susunan data yang sesuai dengan ukuran kernel. Dengan cara ini, operasi *multiply accumulate* pada kernel dapat dilakukan tanpa harus membaca ulang seluruh data citra dari memori utama. Pendekatan seperti ini sejalan dengan implementasi CNN pada FPGA yang menekankan penggunaan *pipeline*, komputasi *fixed-point*, serta pengurangan akses memori eksternal untuk meningkatkan efisiensi sistem [11, 23].

2.2.6 First In First Out (FIFO)

First In First Out atau FIFO adalah struktur penyimpanan data sementara yang bekerja dengan prinsip data yang pertama masuk akan menjadi data pertama yang keluar. FIFO banyak digunakan pada sistem digital untuk mengatur aliran data antarblok, terutama ketika dua bagian sistem tidak selalu bekerja pada kecepatan yang sama [26]. Dalam implementasi FPGA, FIFO dapat digunakan sebagai penyangga data agar proses pengiriman dan penerimaan data tetap stabil meskipun terdapat perbedaan waktu proses antarblok.



Gambar 2.3: Ilustrasi FIFO

Pada arsitektur konvolusi, FIFO dapat digunakan sebagai *line buffer* untuk menyimpan satu atau beberapa baris data citra [11]. Ketika piksel citra masuk secara berurutan, FIFO menahan data dari baris sebelumnya sehingga data tersebut masih tersedia ketika baris berikutnya sedang diproses. Untuk membentuk *window* 3×3 , sistem umumnya membutuhkan dua FIFO sebagai penyangga dua baris sebelumnya. Data dari baris saat ini, baris sebelumnya, dan dua baris sebelumnya kemudian digabungkan dengan shift register untuk menghasilkan sembilan piksel yang dibutuhkan oleh kernel konvolusi.

Hubungan antara FIFO dan shift register dapat dilihat sebagai dua tingkatan penyimpanan yang saling melengkapi. FIFO bekerja pada tingkat baris citra, sedangkan shift register bekerja pada tingkat piksel dalam satu baris [11]. FIFO menyediakan data vertikal dari baris yang berbeda, sedangkan shift register menyusun data horizontal dari beberapa piksel yang berurutan. Dengan kombinasi ini, sistem dapat membentuk *sliding window* tanpa harus mengambil data berulang kali dari memori utama. Hal ini penting karena operasi konvolusi pada CNN membutuhkan akses data yang intensif, sehingga efisiensi penyimpanan sementara dan penggunaan ulang data menjadi bagian penting dalam rancangan akselerator CNN berbasis FPGA [23].

“latex

2.2.7 Konvolusi

Konvolusi merupakan operasi utama pada *Convolutional Neural Network* (CNN) yang digunakan untuk mengekstraksi fitur dari citra masukan. Pada operasi ini, sebuah *kernel* atau *filter* digeser pada area lokal citra untuk menghasilkan nilai fitur baru. Setiap nilai keluaran diperoleh dari hasil perkalian antara elemen citra dan elemen kernel, kemudian seluruh hasil perkalian tersebut dijumlahkan. Dengan mekanisme ini, konvolusi dapat menangkap pola lokal seperti tepi, garis, lengkungan, atau bentuk sederhana yang kemudian dapat digunakan oleh lapisan berikutnya untuk membentuk fitur yang lebih kompleks [27, 28].

Secara matematis, operasi konvolusi dua dimensi pada masukan dengan banyak kanal dapat dituliskan sebagai berikut:

$$y_{k,i,j} = b_k + \sum_{c=0}^{C_{in}-1} \sum_{m=0}^{K_h-1} \sum_{n=0}^{K_w-1} x_{c,iS+m-P,jS+n-P} \cdot w_{k,c,m,n} \quad (2.2)$$

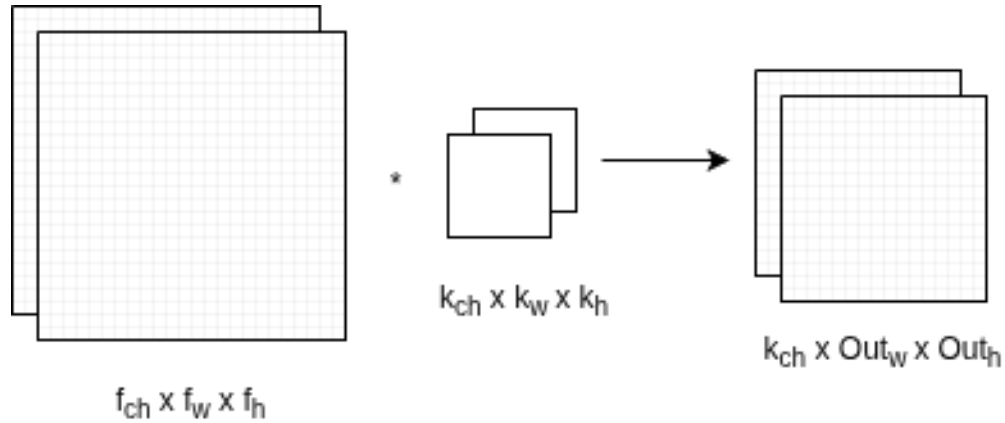
dengan x adalah data masukan, w adalah bobot kernel, b_k adalah bias pada kanal keluaran ke- k , C_{in} adalah jumlah kanal masukan, K_h dan K_w adalah ukuran tinggi dan lebar kernel, S adalah *stride*, dan P adalah *padding*. Indeks i dan j menunjukkan posisi spasial pada *feature map* keluaran.

Ukuran keluaran dari operasi konvolusi dipengaruhi oleh ukuran masukan, ukuran kernel, *stride*, dan *padding*. Jika tinggi dan lebar masukan adalah H_{in} dan W_{in} , maka ukuran keluarannya dapat dihitung dengan:

$$H_{out} = \left\lfloor \frac{H_{in} + 2P - K_h}{S} \right\rfloor + 1 \quad (2.3)$$

$$W_{out} = \left\lfloor \frac{W_{in} + 2P - K_w}{S} \right\rfloor + 1 \quad (2.4)$$

Pada implementasi perangkat keras, persamaan konvolusi tersebut direalisasikan sebagai operasi *multiply accumulate* (MAC), yaitu perkalian antara data dan bobot yang diikuti dengan penjumlahan akumulatif. Karena operasi ini dilakukan berulang pada banyak piksel dan banyak kernel, konvolusi menjadi bagian yang paling dominan dalam beban komputasi CNN.



Gambar 2.4: Operasi Konvolusi

2.2.8 Neural Network

Neural network merupakan model komputasi yang tersusun dari beberapa lapisan neuron buatan. Setiap neuron menerima sejumlah masukan, melakukan operasi penjumlahan berbobot, menambahkan bias, kemudian melewatkan hasilnya ke fungsi aktivasi. Mekanisme ini membuat *neural network* mampu membentuk hubungan non-linear antara data masukan dan keluaran. Dalam konteks klasifikasi citra, lapisan-lapisan tersebut digunakan untuk mengubah data piksel menjadi representasi fitur yang kemudian dipetakan menjadi kelas tertentu [27].

Operasi dasar pada sebuah neuron dapat dituliskan sebagai berikut:

$$z_j^{(l)} = \sum_{i=1}^N w_{j,i}^{(l)} a_i^{(l-1)} + b_j^{(l)} \quad (2.5)$$

$$a_j^{(l)} = \phi \left(z_j^{(l)} \right) \quad (2.6)$$

dengan $a_i^{(l-1)}$ adalah keluaran neuron dari lapisan sebelumnya, $w_{j,i}^{(l)}$ adalah bobot penghubung, $b_j^{(l)}$ adalah bias, $z_j^{(l)}$ adalah hasil penjumlahan berbobot, dan $\phi(\cdot)$ adalah fungsi aktivasi. Dalam bentuk matriks, operasi pada satu lapisan dapat ditulis sebagai:

$$\mathbf{a}^{(l)} = \phi \left(\mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \right) \quad (2.7)$$

Pada proses klasifikasi, keluaran akhir jaringan biasanya merepresentasikan skor untuk masing-masing kelas. Skor tersebut dapat diproses menggunakan fungsi

softmax agar menjadi nilai probabilitas:

$$\hat{y}_k = \frac{e^{z_k}}{\sum_{r=1}^C e^{z_r}} \quad (2.8)$$

dengan $\hat{y}_k$ adalah probabilitas untuk kelas ke- k dan C adalah jumlah kelas. Kelas prediksi kemudian dapat ditentukan dengan mengambil indeks yang memiliki nilai terbesar:

$$\hat{c} = \underset{k}{\operatorname{argmax}}(\hat{y}_k) \quad (2.9)$$

Pada CNN, konsep *neural network* tetap digunakan, tetapi lapisan awal jaringan tidak langsung berbentuk *fully connected*. Lapisan awal CNN menggunakan konvolusi agar proses ekstraksi fitur citra lebih efisien dan tetap mempertahankan informasi spasial dari masukan.

2.2.9 Max Pool

Max pool merupakan operasi reduksi spasial yang digunakan untuk memperkecil ukuran *feature map* dengan cara mengambil nilai maksimum dari area tertentu. Operasi ini tidak memiliki parameter yang dipelajari seperti bobot atau bias, tetapi berperan penting dalam mengurangi ukuran data yang diteruskan ke lapisan berikutnya. Dengan ukuran *feature map* yang lebih kecil, jumlah operasi pada lapisan selanjutnya dapat dikurangi sehingga komputasi menjadi lebih ringan [27, 2].

Secara matematis, operasi *max pool* dapat dituliskan sebagai:

$$y_{c,i,j} = \max_{\substack{0 \leq m < K_h \\ 0 \leq n < K_w}} x_{c,iS+m,jS+n} \quad (2.10)$$

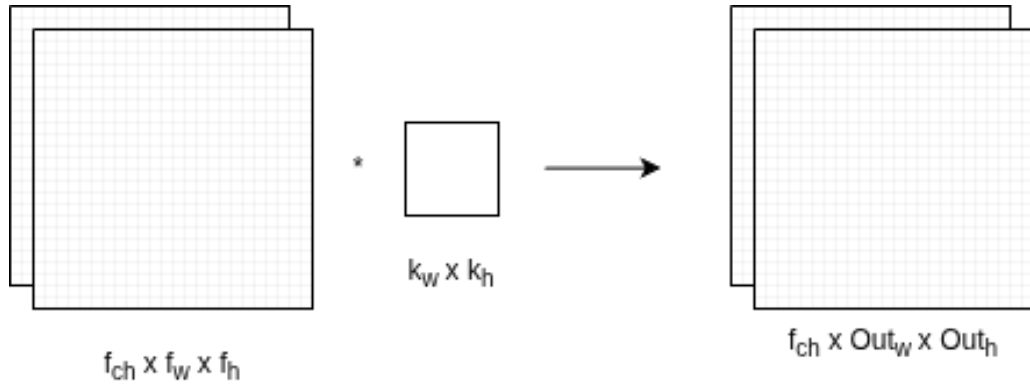
dengan x adalah *feature map* masukan, y adalah *feature map* keluaran, c adalah indeks kanal, K_h dan K_w adalah ukuran jendela pooling, serta S adalah *stride*. Jika digunakan jendela 2×2 dengan *stride* 2, maka ukuran spasial *feature map* akan berkurang menjadi setengah dari ukuran sebelumnya.

Ukuran keluaran dari operasi *max pool* dapat dihitung dengan persamaan yang mirip dengan konvolusi:

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} - K_h}{S} \right\rfloor + 1 \quad (2.11)$$

$$W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} - K_w}{S} \right\rfloor + 1 \quad (2.12)$$

Selain mengurangi ukuran data, *max pool* juga membantu mempertahankan fitur yang paling dominan pada suatu area. Hal ini membuat jaringan lebih toleran terhadap sedikit pergeseran posisi objek pada citra, karena nilai fitur yang kuat tetap dapat dipertahankan meskipun posisinya tidak selalu sama persis.



Gambar 2.5: Operasi Max Pool

2.2.10 Rectified Linear Unit (ReLU)

Rectified Linear Unit (ReLU) merupakan fungsi aktivasi yang banyak digunakan pada CNN karena bentuknya sederhana dan efisien secara komputasi. Fungsi ini akan meneruskan nilai masukan jika bernilai positif, sedangkan nilai negatif akan diubah menjadi nol. Dengan demikian, ReLU memberikan sifat non-linear pada jaringan tanpa membutuhkan operasi matematika yang kompleks seperti eksponensial atau trigonometri [29, 2].

Fungsi ReLU dapat dituliskan sebagai:

$$f(x) = \max(0, x) \quad (2.13)$$

atau dalam bentuk fungsi potongan:

$$f(x) = \begin{cases} x, & x > 0 \\ 0, & x \leq 0 \end{cases} \quad (2.14)$$

Turunan fungsi ReLU dapat dituliskan sebagai:

$$f'(x) = \begin{cases} 1, & x > 0 \\ 0, & x < 0 \end{cases} \quad (2.15)$$

Pada titik $x = 0$, turunan ReLU tidak terdefinisi secara matematis, tetapi pada implementasi jaringan saraf biasanya dapat dipilih sebagai 0 atau 1 sesuai kebutuhan implementasi.

Dalam implementasi perangkat keras, ReLU sangat sederhana karena hanya membutuhkan pembandingan terhadap nol. Jika nilai masukan lebih besar dari nol, maka nilai tersebut diteruskan. Jika nilai masukan lebih kecil atau sama dengan nol, maka keluaran dibuat nol. Kesederhanaan ini membuat ReLU cocok digunakan pada akselerator CNN berbasis FPGA, terutama ketika sistem dirancang untuk menekan penggunaan sumber daya dan menjaga latensi komputasi tetap rendah.

2.2.11 CNN

Convolutional Neural Network (CNN) merupakan pengembangan dari *neural network* yang dirancang khusus untuk mengolah data berbentuk grid, seperti citra digital. CNN memanfaatkan operasi konvolusi untuk mengekstraksi fitur lokal dari citra, kemudian menggabungkannya melalui beberapa lapisan agar jaringan dapat mengenali pola yang lebih kompleks. Pada lapisan awal, CNN biasanya menangkap fitur sederhana seperti tepi atau garis. Pada lapisan yang lebih dalam, fitur tersebut dapat berkembang menjadi bentuk, tekstur, atau pola yang lebih spesifik terhadap kelas tertentu [28, 27].

Secara umum, alur komputasi CNN dapat dipandang sebagai komposisi dari beberapa fungsi lapisan:

$$\mathbf{y} = f_{\theta}(\mathbf{x}) = g_L(g_{L-1}(\cdots g_2(g_1(\mathbf{x})) \cdots)) \quad (2.16)$$

dengan $\mathbf{x}$ adalah citra masukan, $\mathbf{y}$ adalah keluaran jaringan, g_l adalah operasi pada lapisan ke- l , dan θ adalah seluruh parameter yang dipelajari, seperti bobot dan bias. Lapisan pada CNN umumnya terdiri dari kombinasi konvolusi, fungsi aktivasi, *pooling*, dan *fully connected layer*. Kombinasi tersebut membuat CNN dapat melakukan ekstraksi fitur sekaligus klasifikasi dalam satu rangkaian komputasi.

Keunggulan CNN dibandingkan *fully connected neural network* biasa adalah adanya konsep *local connectivity* dan *weight sharing*. *Local connectivity* berarti neuron hanya terhubung pada area lokal dari citra, sedangkan *weight sharing* berarti kernel yang sama digunakan berulang pada banyak posisi. Kedua konsep ini membuat jumlah parameter CNN lebih efisien dibandingkan jika seluruh piksel langsung dihubungkan ke neuron pada lapisan berikutnya. Hal ini menjadi salah satu alasan

CNN banyak digunakan pada tugas pengenalan citra dan klasifikasi objek [2].

Pada penelitian ini, CNN digunakan sebagai model utama untuk melakukan klasifikasi citra. Operasi yang paling dominan pada CNN adalah konvolusi karena melibatkan banyak operasi perkalian dan penjumlahan. Oleh karena itu, ketika CNN diimplementasikan pada FPGA, bagian konvolusi menjadi fokus utama untuk dioptimalkan melalui pemetaan operasi MAC, penggunaan format bilangan *fixed-point*, dan pemanfaatan paralelisme perangkat keras. Dengan pendekatan tersebut, CNN dapat dijalankan sebagai akselerator perangkat keras yang lebih sesuai untuk sistem *embedded vision* dan aplikasi *real-time*.

2.2.12 Kuantitasi

Kuantitasi merupakan proses mengubah representasi numerik dari presisi tinggi menjadi presisi yang lebih rendah. Pada CNN, parameter seperti bobot, bias, dan aktivasi pada awalnya direpresentasikan dalam format *floating-point* karena format tersebut fleksibel untuk proses pelatihan [23]. Namun, format *floating-point* tidak efisien untuk implementasi perangkat keras karena membutuhkan rangkaian aritmetika yang lebih kompleks, ukuran data yang lebih besar, serta konsumsi sumber daya yang lebih tinggi. Oleh karena itu, kuantitasi menjadi tahap penting untuk mengubah model CNN agar lebih sesuai dengan karakteristik FPGA.

Pada implementasi CNN berbasis FPGA, kuantitasi berperan langsung terhadap efisiensi memori, bandwidth, jumlah sumber daya aritmetika, dan latensi inferensi. Jacob et al. menunjukkan bahwa inferensi CNN dapat dijalankan dengan aritmetika integer melalui skema kuantitasi yang tetap menjaga akurasi model [30]. Gupta et al. menunjukkan bahwa jaringan saraf dapat bekerja menggunakan representasi *fixed-point* 16-bit dengan degradasi akurasi yang kecil ketika proses pembulatan dilakukan secara tepat [31]. Lin et al. kemudian membahas kuantitasi *fixed-point* pada jaringan konvolusional dengan menekankan pemilihan *bit-width* yang sesuai pada setiap layer [32]. Pada konteks FPGA, Goyal et al. dan Solovyev et al. memperkuat bahwa perubahan dari *floating-point* menuju *fixed-point* dapat menurunkan kebutuhan komputasi dan memori sehingga model lebih layak diimplementasikan pada platform *embedded* dan akselerator perangkat keras [33, 11]. Dengan demikian, kuantitasi tidak hanya dilihat sebagai proses kompresi model, tetapi sebagai bagian dari strategi perancangan arsitektur agar operasi CNN dapat dipetakan secara efisien ke dalam jalur data FPGA.

Kuantitasi *int8* berfokus pada pemetaan nilai real ke bilangan integer 8-bit

menggunakan faktor skala dan *zero-point*. Jacob et al. menggunakan skema ini untuk mendukung inferensi berbasis aritmetika integer, sedangkan Krishnamoorthi menekankan bahwa kuantitasi 8-bit pada bobot dan aktivasi dapat menurunkan ukuran model serta mempercepat inferensi pada perangkat yang mendukung operasi integer [30, 34].

$$S = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}} \quad (2.17)$$

$$Z = \text{round} \left(q_{\min} - \frac{x_{\min}}{S} \right) \quad (2.18)$$

$$q_8 = \text{clip} \left(\text{round} \left(\frac{x}{S} \right) + Z, q_{\min}, q_{\max} \right) \quad (2.19)$$

$$\hat{x}_8 = S(q_8 - Z) \quad (2.20)$$

Kuantitasi *fixed-point* 16-bit menggunakan posisi bit pecahan yang tetap, sehingga nilai real dikonversi menjadi bilangan integer bertanda dengan faktor skala 2^F . Gupta et al. membuktikan bahwa representasi 16-bit *fixed-point* dapat digunakan pada jaringan saraf dengan hasil yang tetap kompetitif, sedangkan Lin et al. dan Goyal et al. menunjukkan pentingnya pemilihan jumlah bit dan posisi pecahan agar kesalahan kuantitasi tidak merusak akurasi model [31, 32, 33]. Pada format ini, F menyatakan jumlah bit pecahan yang digunakan. Proses kuantitasi dan pendekatan kembali ke nilai real dapat dituliskan sebagai berikut.

$$q_{16} = \text{clip} \left(\text{round}(x \cdot 2^F), -2^{15}, 2^{15} - 1 \right) \quad (2.21)$$

$$\hat{x}_{16} = q_{16} \cdot 2^{-F} \quad (2.22)$$

Untuk parameter CNN, bobot, aktivasi, dan bias dapat dikonversi menggunakan jumlah bit pecahan yang disesuaikan dengan kebutuhan masing-masing parameter.

$$q_w = \text{clip} \left(\text{round}(w \cdot 2^{F_w}), -2^{15}, 2^{15} - 1 \right) \quad (2.23)$$

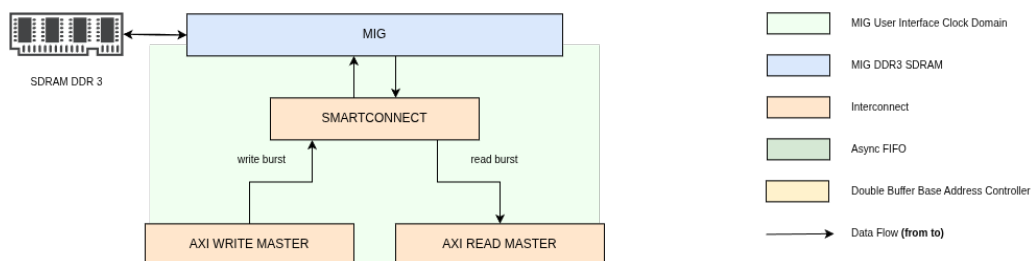
$$q_a = \text{clip} \left(\text{round}(a \cdot 2^{F_a}), -2^{15}, 2^{15} - 1 \right) \quad (2.24)$$

$$q_b = \text{clip} \left(\text{round}(b \cdot 2^{F_b}), -2^{31}, 2^{31} - 1 \right) \quad (2.25)$$

Trade-off utama antara *int8* dan *fixed-point* 16-bit terletak pada keseimbangan antara efisiensi dan kestabilan numerik. *Int8* memberikan ukuran data yang lebih kecil, kebutuhan memori yang lebih rendah, dan potensi bandwidth yang lebih hemat, tetapi membutuhkan pemilihan skala, *zero-point*, dan proses kalibrasi yang lebih hati-hati agar akurasi tidak turun secara signifikan. Sebaliknya, *fixed-point* 16-bit membutuhkan sumber daya yang lebih besar dibandingkan *int8*, tetapi memberikan rentang dan ketelitian yang lebih aman untuk menjaga hasil komputasi tetap dekat dengan model *floating-point*. Pada penelitian ini, *fixed-point* 16-bit dipilih karena memberikan titik tengah yang kuat antara efisiensi implementasi FPGA dan kestabilan akurasi inferensi.

2.2.13 Double Data Rate (DDR)

Double Data Rate atau DDR merupakan memori eksternal yang digunakan untuk menyediakan kapasitas penyimpanan lebih besar dibandingkan memori internal FPGA. Pada sistem berbasis citra, kebutuhan memori menjadi penting karena data yang diproses tidak hanya berupa satu piksel, tetapi dapat berupa satu frame penuh atau blok data berukuran besar. Untuk citra RGB565 beresolusi 640×480 , satu frame membutuhkan kapasitas penyimpanan yang jauh lebih besar dibandingkan penyimpanan lokal yang idealnya digunakan untuk buffer kecil, register, atau blok komputasi. Oleh karena itu, DDR ditempatkan sebagai *frame buffer* utama, sedangkan BRAM digunakan untuk menyimpan data sementara pada jalur pemrosesan yang lebih dekat dengan logika FPGA. Gambar 2.6 menunjukkan posisi DDR pada sistem kamera, AXI, MIG, dan VGA.



Gambar 2.6: Ilustrasi hubungan AXI, MIG, dan DDR pada sistem pemindahan data berbasis memori eksternal.

Advanced eXtensible Interface atau AXI merupakan protokol antarmuka yang digunakan untuk menghubungkan modul komputasi dengan subsistem memori atau periferan pada sistem digital. AXI mendukung transaksi *read* dan *write* secara terpisah, memiliki kanal alamat, data, dan respons, serta memungkinkan transfer data dalam bentuk *burst*. Karakter ini membuat AXI sesuai untuk pemindahan data berukuran besar karena beberapa data dapat dikirim atau diambil secara berurutan tanpa harus membentuk alamat baru untuk setiap data. Pada sistem pemrosesan citra, AXI dapat digunakan untuk mengirim data ke memori melalui AXI *write* master dan mengambil kembali data dari memori melalui AXI *read* master. Jika diperlukan, buffer lokal seperti FIFO atau BRAM dapat ditempatkan di sisi modul untuk menyesuaikan laju data, tetapi mekanisme utama pemindahan data tetap dilakukan melalui transaksi AXI [35].

Memory Interface Generator atau MIG merupakan IP yang digunakan sebagai pengendali akses DDR pada FPGA. DDR tidak dapat diakses secara langsung seperti register biasa karena memori ini memiliki aturan waktu, proses inisialisasi, kalibrasi, pengaturan alamat, dan sinyal fisik yang lebih kompleks. MIG menyederhanakan bagian tersebut dengan menyediakan antarmuka yang dapat digunakan oleh logika FPGA, salah satunya melalui AXI. Dengan adanya MIG, modul logika tidak perlu mengatur sinyal fisik DDR secara langsung. Modul cukup membentuk transaksi baca atau tulis melalui antarmuka AXI, sedangkan MIG menerjemahkan transaksi tersebut menjadi operasi memori yang sesuai dengan protokol DDR. Peran ini membuat MIG menjadi penghubung penting antara logika digital di dalam FPGA dan memori eksternal yang memiliki aturan akses lebih ketat [36].

Penggunaan DDR pada akselerator CNN dilakukan pada penelitian yang membutuhkan penyimpanan data berukuran besar dan akses memori yang terstruktur. Solovyev et al. menggunakan memori eksternal sebagai bagian dari sistem pemrosesan video real-time, sementara memori lokal tetap digunakan untuk mendukung aliran data yang dekat dengan blok komputasi [11]. Zhang et al. menggunakan DDR3 sebagai penyimpanan data utama pada akselerator CNN berbasis FPGA, kemudian mengoptimalkan akses data menggunakan *loop tiling* dan pemanfaatan BRAM agar bandwidth memori dapat digunakan secara lebih efisien [23]. Kedua penelitian tersebut menunjukkan bahwa DDR berperan sebagai penyimpanan eksternal ketika ukuran data melebihi kapasitas memori lokal FPGA, sedangkan memori internal digunakan untuk mempercepat akses data yang sedang aktif diproses.

2.2.14 FPGA ARTIX 7

FPGA Artix-7 merupakan keluarga FPGA dari Xilinx yang dirancang untuk memberikan keseimbangan antara performa, konsumsi daya, dan biaya implementasi. Pada penelitian ini, board yang digunakan adalah Arty A7-100T dengan perangkat FPGA XC7A100T. Board ini menyediakan sumber daya logika, memori internal, DSP, dan DDR eksternal yang cukup untuk membangun sistem akselerator CNN skala embedded. Spesifikasi utama Arty A7-100T ditunjukkan pada Tabel 2.1.

Tabel 2.1: Spesifikasi utama Arty A7-100T

Komponen	Spesifikasi
FPGA	Xilinx Artix-7 XC7A100T
LUT	63400
Flip-Flop	126800
Block RAM	4860 Kb
DSP Slice	240 DSP48E1
DDR	256 MB DDR3L

Jika dibandingkan dengan penelitian sebelumnya, Arty A7-100T berada pada kelas menengah. Board ini memiliki sumber daya lebih besar dibandingkan FPGA lainnya seperti Spartan-6 atau Cyclone IV yang digunakan pada penelitian Solovyev et al., tetapi masih lebih terbatas dibandingkan board kelas tinggi seperti Virtex-7 VC707 yang digunakan pada penelitian Zhang et al. [11, 23]. Kondisi ini membuat Arty A7-100T cukup mampu untuk implementasi CNN *accelerator*, karena desain harus benar-benar memperhatikan pemakaian resource. Operasi MAC dapat dipetakan ke DSP48E1, data sementara dapat disimpan pada BRAM, sedangkan FF dan LUT digunakan untuk membangun jalur kendali, pipeline, FSM, serta register antarblok.

Dari sisi daya dan latensi, FPGA Artix-7 mendukung pendekatan yang lebih hemat daya dibandingkan komputasi umum karena jalur data dapat dibuat khusus sesuai kebutuhan algoritma. Penggunaan komputasi *fixed-point*, DSP untuk operasi MAC, serta pipeline dapat menurunkan latensi komputasi karena operasi tidak perlu dijalankan sebagai instruksi perangkat lunak satu per satu. Namun, performa akhir tetap bergantung pada keseimbangan antara jumlah operasi paralel, akses DDR, dan penggunaan *time multiplexing*. Semakin banyak operasi dibuat paralel, latensi dapat ditekan, tetapi konsumsi resource dan daya dinamis meningkat. Sebaliknya, pemakaian ulang resource dapat menghemat DSP dan LUT, tetapi waktu komputasi menjadi

lebih panjang.

2.2.15 OV7670

OV7670 merupakan sensor kamera CMOS beresolusi VGA yang dapat menghasilkan citra hingga ukuran 640×480 piksel. Sensor ini banyak digunakan pada implementasi FPGA karena menyediakan keluaran data paralel dan sinyal sinkronisasi yang relatif mudah dihubungkan dengan logika digital. Konfigurasi internal kamera dilakukan melalui Serial Camera Control Bus atau SCCB, sedangkan data citra dikeluarkan melalui jalur data paralel yang disertai sinyal *pixel clock*, HREF, dan VSYNC [12].

Pada aliran data kamera, sinyal VSYNC digunakan sebagai penanda pergantian frame, HREF digunakan untuk menunjukkan bahwa data pada satu baris sedang valid, dan *pixel clock* digunakan sebagai acuan pengambilan data piksel. OV7670 dapat menghasilkan beberapa format keluaran, seperti YUV, RGB, dan RGB565. Format RGB565 sering digunakan pada sistem FPGA karena satu piksel dapat direpresentasikan dalam 16-bit, yaitu 5-bit merah, 6-bit hijau, dan 5-bit biru. Format ini cukup ringan untuk disimpan dan diproses, tetapi tetap mampu menampilkan warna yang memadai untuk sistem embedded vision.

Dalam sistem CNN berbasis kamera, OV7670 berperan sebagai sumber data visual yang masuk ke sistem secara kontinu. Hal ini membuat logika penerima kamera harus mampu membaca data berdasarkan sinyal sinkronisasi, menyusun data piksel dengan benar, dan menjaga agar aliran data tidak terputus. Karena data kamera datang mengikuti domain *pixel clock*, rancangan FPGA perlu memperhatikan sinkronisasi, buffering, dan validitas data sebelum data tersebut digunakan untuk pemrosesan atau ditampilkan kembali.

2.2.16 VGA

Video Graphics Array atau VGA merupakan antarmuka tampilan analog yang mengirimkan sinyal warna merah, hijau, dan biru bersama sinyal sinkronisasi horizontal dan vertikal. Pada FPGA, tampilan VGA dibentuk dengan menghasilkan data warna RGB dan sinyal timing yang sesuai dengan resolusi layar. Untuk resolusi 640×480 , sistem perlu menghasilkan urutan piksel, periode *blanking*, sinyal HSYNC, dan sinyal VSYNC agar monitor dapat menampilkan gambar dengan stabil.

Pada penelitian ini, keluaran VGA dapat dibantu menggunakan Pmod VGA

dari Digilent. Pmod VGA menyediakan jalur warna 12-bit, yaitu 4-bit untuk merah, 4-bit untuk hijau, dan 4-bit untuk biru, serta dua sinyal sinkronisasi standar berupa HSYNC dan VSYNC [37]. Modul ini mempermudah FPGA untuk menghasilkan keluaran VGA karena konversi sinyal digital RGB menuju level analog dilakukan melalui rangkaian resistor pada Pmod. Dengan demikian, FPGA cukup menghasilkan data warna digital dan sinyal sinkronisasi yang sesuai dengan timing VGA.

Dalam sistem embedded vision, VGA berfungsi sebagai jalur keluaran visual untuk menampilkan hasil pembacaan kamera atau hasil pemrosesan citra. Keberadaan VGA membuat sistem dapat diamati secara langsung tanpa harus selalu mengirim data ke komputer. Dari sisi perancangan, bagian VGA menuntut aliran data yang stabil karena tampilan layar bergerak mengikuti timing yang tetap. Jika data piksel tidak tersedia pada saat dibutuhkan, tampilan dapat mengalami gangguan seperti warna tidak sesuai, gambar bergeser, atau frame terlihat tidak stabil.

2.3 Analisis Perbandingan Metode

Pada penelitian sebelumnya menunjukkan bahwa hasil inferensi CNN pada FPGA dapat diselesaikan sebelum satu frame dapat selesai terbentuk. Camera OV7670 memiliki 30 FPS [12]. Sedangkan latensi terbesar didapat pada nilai 150 FPS [11]. Sedangkan konsumsi daya pada FPGA relatif lebih rendah dibandingkan dengan CPU dan GPU. Efisiensi daya ini pada penelitian yang dilakukan oleh Qasimeh et al. dan Guo et al. [9, 38]. Hal ini menunjukkan bahwa FPGA cocok untuk digunakan dalam implementasi CNN dengan daya lebih rendah.

Tabel 2.2: Perbandingan implementasi sistem pada penelitian terkait

Penelitian	CNN	Kamera	VGA	Board/Platform
Solovyev et al. [11]	Ya	Ya	Ya	<i>DE0-Nano & Intel Cyclone IV</i>
Qiu et al. [13]	Ya	–	–	<i>Xilinx ZC706</i>
Dua et al. [14]	Ya	–	–	<i>Arria 10 GX1150 & Stratix 10 GX2800</i>
Navaneethan et al. [15]	–	Ya	Ya	<i>Spartan-6 & Artix-7</i>
Sun et al. [16]	–	Ya	Ya	<i>FPGA MicroBlaze based</i>

Berdasarkan Tabel 2.2, penelitian terkait dapat dikelompokkan menjadi dua arah utama. Penelitian seperti Solovyev et al. sudah menunjukkan implementasi inferensi CNN secara *end-to-end* pada FPGA, sedangkan Qiu et al. dan Dua et al. lebih menekankan optimasi akselerator CNN dari sisi komputasi model, arsitektur paralel, dan efisiensi operasi. Di sisi lain, Navaneethan et al. dan Sun et al. lebih

berfokus pada integrasi sistem citra, yaitu pengambilan data dari kamera OV7670 dan penampilan citra melalui VGA, tetapi belum membahas akselerasi inferensi CNN. Hal ini menunjukkan adanya celah penelitian pada integrasi antara sistem akuisisi citra, tampilan video, dan akselerator CNN dalam satu platform FPGA. Oleh karena itu, penelitian ini tidak hanya berfokus pada implementasi CNN berbasis *fixed-point*, tetapi juga pada bagaimana sistem tersebut dapat diintegrasikan dengan periferal citra agar lebih mendekati kebutuhan aplikasi *embedded vision* secara langsung.

Tabel 2.3: Perbandingan performa penelitian terkait

Penelitian	Latensi/FPS	Arsitektur Model	Daya (W)	GOPS	Loss	Akurasi
[11]	>150 FPS	CNN	–	–	–	Tidak berubah setelah kuantisasi
[13]	224.60 ms	VGG16-SVD	9.63	137	–	86.66%
[14]	140ms	AlexNet	2,1	–	–	–

Selain aspek integrasi sistem, perbandingan performa pada Tabel 2.3 juga menunjukkan bahwa konsumsi daya menjadi faktor penting dalam implementasi CNN. Pada penelitian Qiu et al., implementasi VGG16-SVD pada FPGA hanya membutuhkan daya sebesar 9,63 W, sedangkan platform CPU dan GPU yang digunakan sebagai pembanding memiliki konsumsi daya yang jauh lebih tinggi, yaitu 135 W untuk CPU dan 250 W untuk GPU [13]. Meskipun GPU mampu memberikan performa komputasi yang lebih besar, konsumsi dayanya mencapai sekitar 26 kali lebih tinggi dibandingkan FPGA. Kondisi ini memperlihatkan bahwa CPU dan GPU tidak selalu menjadi pilihan paling efisien untuk sistem *real-time* dan *embedded*, terutama ketika batasan daya dan latensi menjadi pertimbangan utama. Dengan demikian, FPGA menjadi alternatif yang relevan karena dapat menyediakan jalur komputasi khusus melalui paralelisme, *pipeline*, dan representasi *fixed-point* sehingga proses inferensi dapat dilakukan dengan konsumsi daya yang lebih rendah.

BAB III

METODOLOGI PENELITIAN

3.1 Alat dan Bahan

Alat dan bahan yang digunakan pada penelitian ini terbagi atas perangkat keras dan perangkat lunak yang akan dijelaskan seperti berikut.

3.1.1 Perangkat Keras

Perangkat keras terdiri dari sebuah FPGA board yang digunakan sebagai media inferensi model CNN serta sejumlah peripheral media untuk melakukan *streaming media* untuk pengambilan data. Detail mengenai perangkat keras yang digunakan terdapat pada daftar berikut.

- a. Digilent Arty A7-100T,
- b. OV7670,
- c. VGA Pmod Digilent,
- d. USB,
- e. 14 Jumper Male-Female,

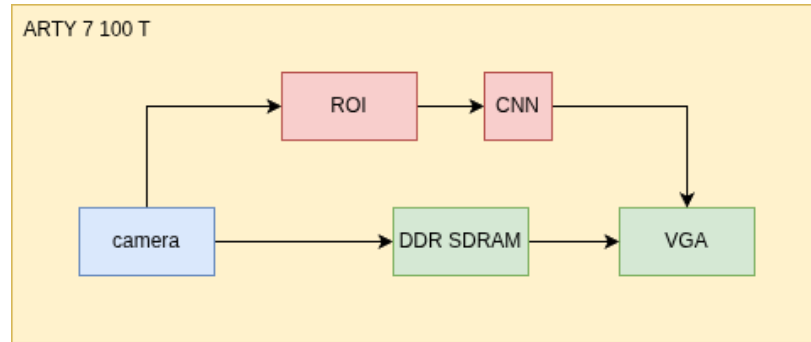
3.1.2 Perangkat Lunak

Perangkat lunak yang digunakan selama proses pengembangan mencakup lingkungan kerja dan lingkungan uji. Detail mengenai perangkat lunak yang digunakan terdapat pada daftar berikut.

- a. Software VIVADO,
- b. Linux Operating System,
- c. Python,
- d. Kaggle,
- e. Verilog Icairus,
- f. Visual Studio Code.

3.2 Rancangan Sistem

3.2.1 Rancangan Umum Sistem



Gambar 3.1: Rancangan Umum Sistem

DDR SDRAM digunakan sebagai *frame buffer* agar data citra dari kamera dan data yang akan ditampilkan ke VGA dapat dipisahkan secara lebih stabil. Jalur kamera, DDR, dan VGA berperan sebagai sistem akuisisi dan tampilan citra, sedangkan jalur ROI dan CNN berperan sebagai sistem klasifikasi. Pemisahan ini dilakukan agar proses inferensi CNN tidak harus dilakukan pada seluruh frame, melainkan hanya pada area tertentu yang dianggap relevan. Dengan demikian, beban komputasi dapat dikurangi tanpa menghilangkan fungsi utama sistem sebagai sistem klasifikasi citra berbasis FPGA.

3.2.2 Rancangan Arsitektur CNN

Arsitektur CNN yang digunakan pada penelitian ini dirancang sebagai model klasifikasi citra berukuran kecil agar sesuai dengan keterbatasan sumber daya pada FPGA. Input model berupa citra grayscale berukuran 28×28 piksel yang diperoleh dari hasil pemrosesan ROI. Model CNN dikembangkan terlebih dahulu menggunakan Python sebagai model acuan sebelum parameter bobot dan biasnya dipindahkan ke implementasi Verilog. Penggunaan CNN dipilih karena metode ini mampu mengekstraksi fitur citra secara bertahap melalui operasi konvolusi, aktivasi, dan pooling [27, 28].

Secara umum, model terdiri dari tiga blok konvolusi dan dua lapisan *fully connected*. Blok pertama menerima satu kanal input dan menghasilkan 8 kanal fitur. Blok kedua menghasilkan 16 kanal fitur, sedangkan blok ketiga menghasilkan 32

kanal fitur. Setiap blok konvolusi menggunakan kernel 3×3 yang diikuti oleh fungsi aktivasi ReLU dan proses *max pooling*. Setelah tahap ekstraksi fitur selesai, hasil akhir diproses oleh lapisan *fully connected* untuk menghasilkan 10 nilai keluaran yang merepresentasikan kelas prediksi. Pada implementasi hardware, tahap *softmax* tidak digunakan karena proses klasifikasi cukup dilakukan dengan membandingkan nilai keluaran terbesar menggunakan *argmax*.

Arsitektur ini dipilih karena memiliki jumlah parameter yang relatif kecil, tetapi tetap mampu merepresentasikan proses ekstraksi fitur CNN secara bertingkat. Selain itu, ukuran model yang tidak terlalu besar membuatnya lebih memungkinkan untuk diimplementasikan pada FPGA dengan komputasi *fixed-point*. Parameter model berupa bobot dan bias disimpan sebagai data konstan pada memori internal, sehingga proses inferensi dapat dilakukan secara langsung oleh rangkaian hardware tanpa memerlukan proses pelatihan ulang di dalam FPGA.

3.2.3 Rancangan Representasi Fixed-Point

Representasi numerik pada rancangan akselerator CNN dibuat menggunakan format *fixed-point* 16-bit. Pemilihan format ini dilakukan karena model CNN pada tahap pelatihan masih menggunakan *floating-point*, sedangkan implementasi pada FPGA lebih sesuai apabila operasi aritmetika direalisasikan dalam bentuk bilangan integer atau *fixed-point*. Operasi *floating-point* membutuhkan rangkaian aritmetika yang lebih kompleks dan dapat meningkatkan penggunaan resource, terutama pada operasi perkalian dan akumulasi yang muncul berulang pada layer konvolusi dan *fully connected*. Oleh karena itu, parameter model hasil pelatihan, seperti bobot dan bias, dikonversi terlebih dahulu ke dalam format *fixed-point* sebelum dimasukkan ke dalam memori parameter pada Verilog [23, 11].

Pada penelitian ini, representasi *fixed-point* menggunakan jumlah bit pecahan atau *fractional bit* sebesar $F = 14$. Dengan konfigurasi tersebut, nilai *floating-point* dikalikan dengan faktor skala 2^{14} , kemudian dibulatkan dan dibatasi agar tetap berada pada rentang bilangan bertanda 16-bit. Pemakaian *fractional bit* ini penting karena nilai bobot CNN umumnya berada pada rentang kecil, sehingga dibutuhkan resolusi pecahan yang cukup halus agar informasi parameter tidak banyak hilang setelah proses kuantisasi. Secara umum, proses konversi dari nilai real x ke representasi *fixed-point* 16-bit dapat dituliskan sebagai berikut.

$$q_{16} = \text{clip}(\text{round}(x \cdot 2^F), -2^{15}, 2^{15} - 1) \quad (3.1)$$

$$\hat{x}_{16} = q_{16} \cdot 2^{-F} \quad (3.2)$$

Pada Persamaan 3.1 dan 3.2, fungsi $\text{round}(\dots)$ digunakan untuk membulatkan hasil perkalian skala ke bilangan integer terdekat, sedangkan fungsi $\text{clip}(\dots)$ digunakan untuk mencegah nilai keluar dari rentang 16-bit bertanda. Dengan $F = 14$, nilai yang disimpan memiliki resolusi sebesar 2^{-14} . Artinya, perubahan kecil pada nilai bobot masih dapat direpresentasikan dengan cukup baik. Konfigurasi ini dipilih sebagai titik tengah antara ketelitian numerik dan efisiensi hardware, karena jumlah bit pecahan yang terlalu kecil dapat memperbesar galat kuantisasi, sedangkan jumlah bit pecahan yang terlalu besar dapat mempersempit rentang nilai integer yang dapat direpresentasikan [32].

Konversi parameter pada setiap layer dilakukan dengan prinsip yang sama [31]. Bobot hasil pelatihan dikonversi menjadi q_w , aktivasi dikonversi menjadi q_a , dan bias dikonversi menjadi q_b . Pada rancangan ini, bobot dan bias disimpan dalam format 16-bit bertanda, sedangkan hasil perkalian dan akumulasi pada jalur MAC dibuat lebih lebar agar tidak langsung mengalami *overflow*. Proses kuantisasi masing-masing parameter dapat dituliskan sebagai berikut.

Proses translasi parameter dari Python ke Verilog dilakukan dengan mengubah nilai *floating-point* hasil pelatihan menjadi data heksadesimal *fixed-point*. Kode translasi tersebut pada dasarnya melakukan tiga tahap utama, yaitu mengalikan nilai parameter dengan 2^F , membulatkan hasilnya ke bilangan integer, kemudian membatasi nilai agar tetap berada pada rentang 16-bit bertanda. Hasil konversi selanjutnya disimpan ke dalam file memori seperti `conv1_w.mem`, `conv1_b.mem`, `conv2_w.mem`, `conv2_b.mem`, dan file parameter lain yang digunakan oleh modul Verilog. Dengan cara ini, parameter yang digunakan pada simulasi Python dan implementasi Verilog tetap berasal dari model yang sama, tetapi sudah berada dalam bentuk numerik yang sesuai untuk jalur data FPGA.

3.2.4 Rancangan Akselerator CNN pada FPGA

Akselerator CNN pada FPGA dirancang untuk menjalankan proses inferensi secara langsung pada hardware. Operasi utama pada CNN adalah konvolusi, yaitu proses perkalian antara data input dengan bobot kernel yang kemudian dijumlahk-

an. Oleh karena itu, bagian utama dari akselerator CNN direalisasikan menggunakan struktur *multiply accumulate* (MAC). Operasi MAC dipetakan ke DSP pada FPGA karena DSP lebih sesuai untuk menjalankan operasi perkalian dan akumulasi dibandingkan apabila seluruh operasi dilakukan menggunakan LUT. Namun, karena jumlah DSP pada FPGA terbatas, rancangan akselerator tidak dibuat paralel penuh, melainkan menggunakan kombinasi paralelisme dan *time multiplexing* [21, 22]. Target pemetaan resource pada masing-masing bagian CNN ditunjukkan pada Tabel 3.1.

Tabel 3.1: Target penggunaan resource pada implementasi CNN

Layer CNN	Input	Output	Target DSP	Strategi Implementasi
Conv 1	1 <i>channel</i>	8 <i>channel</i>	72	Paralel untuk 8 output <i>channel</i>
Conv 2	8 <i>channel</i>	16 <i>channel</i>	72	<i>Time multiplexing</i> 16 output <i>channel</i>
Conv 3	16 <i>channel</i>	32 <i>channel</i>	72	<i>Time multiplexing</i> 32 output <i>channel</i>
Dense	32 <i>feature</i>	16 <i>feature</i>	8	MAC digunakan bergantian pada pada setiap neuron
Dense	16 <i>feature</i>	10 kelas	10	MAC digunakan bergantian pada pada setiap neuron

Pada rancangan ini, blok konvolusi pertama dibuat lebih paralel karena jumlah kanal input dan output masih kecil. Sementara itu, blok konvolusi kedua dan ketiga menggunakan *time multiplexing* agar jumlah DSP yang digunakan tetap dapat dikendalikan. Conv2 menghitung beberapa output *channel* secara bergantian menggunakan kelompok MAC yang sama. Conv3 juga menggunakan pendekatan serupa, tetapi kanal input dibagi menjadi dua kelompok sehingga satu kelompok MAC dapat dipakai ulang untuk menyelesaikan seluruh output *channel*. Strategi ini membuat jumlah DSP yang digunakan lebih rendah dibandingkan implementasi paralel penuh, walaupun membutuhkan jumlah siklus yang lebih banyak untuk menyelesaikan satu keluaran fitur. Pendekatan ini sejalan dengan konsep *folding* pada akselerator CNN FPGA, yaitu penggunaan resource komputasi yang lebih sedikit untuk menjalankan

operasi yang lebih besar secara bergantian [24].

Selain MAC, bagian penting dari akselerator CNN adalah pembentukan window konvolusi. Karena operasi konvolusi 3×3 membutuhkan sekumpulan data piksel atau fitur yang berdekatan, maka digunakan *shift register* dan *line buffer* untuk membentuk window tersebut. Data input tidak dibaca ulang dari awal untuk setiap operasi konvolusi, tetapi digeser secara berurutan sehingga window baru dapat terbentuk dari data yang sudah tersimpan pada buffer baris sebelumnya. Penggunaan *shift register* dan *line buffer* ini membantu mengurangi kebutuhan akses memori dan membuat proses konvolusi lebih sesuai dengan aliran data streaming pada FPGA. Pendekatan serupa juga digunakan pada beberapa akselerator CNN berbasis FPGA, seperti FINN melalui *Sliding Window Unit* dan AoCStream melalui arsitektur *stream-based line-buffer* [24, 39].

Setiap blok CNN dikendalikan menggunakan *finite state machine* (FSM) [40]. FSM digunakan untuk mengatur urutan kerja pada masing-masing blok, seperti menunggu data masuk, menjalankan proses komputasi, mengumpulkan hasil perhitungan, dan mengirimkan data ke blok berikutnya. Pada blok yang menggunakan *time multiplexing*, FSM berperan penting karena satu unit komputasi digunakan secara bergantian untuk beberapa *channel*. Dengan demikian, FSM memastikan bahwa proses pengambilan data, pemilihan bobot, akumulasi hasil, dan pengiriman output dilakukan pada urutan yang benar [41].

Untuk menjaga kestabilan aliran data antarblok, rancangan akselerator menggunakan FIFO dan mekanisme *valid-ready*. FIFO digunakan sebagai penyangga antara blok yang memiliki kecepatan proses berbeda. Hal ini diperlukan karena tidak semua blok CNN menghasilkan data pada jumlah siklus yang sama. Misalnya, blok konvolusi yang menggunakan *time multiplexing* membutuhkan waktu lebih lama dibandingkan blok pooling atau blok komparator [42]. Dengan adanya FIFO, blok sebelumnya tetap dapat mengirim data selama FIFO belum penuh, sedangkan blok berikutnya dapat mengambil data ketika sudah siap. Mekanisme FIFO seperti ini umum digunakan pada desain hardware berbasis aliran data untuk memisahkan proses produksi dan konsumsi data [26]. Aliran data dengan mekanisme *valid-ready* ditunjukkan pada Gambar 2.1.

Mekanisme *valid-ready* digunakan sebagai bentuk *backpressure*. Sinyal *valid* menunjukkan bahwa data dari blok sebelumnya sudah tersedia, sedangkan sinyal *ready* menunjukkan bahwa blok berikutnya siap menerima data. Perpindahan data hanya terjadi ketika kedua sinyal tersebut aktif secara bersamaan. Dengan cara ini,

counter, *shift register*, FIFO, dan FSM hanya bergerak ketika data benar-benar berhasil diterima oleh blok berikutnya. Hal ini mencegah terjadinya kehilangan data atau pergeseran urutan fitur ketika salah satu blok sedang sibuk.

Data antarblok pada rancangan ini tetap menggunakan bus yang cukup lebar untuk menjaga hasil antara dari proses *fixed-point*. Namun, sebelum data aktivasi masuk ke operasi MAC pada layer berikutnya, lebar aktivasi dibatasi agar operasi perkalian antara aktivasi dan bobot 16-bit tetap efisien pada DSP. Dengan demikian, hasil akumulasi MAC tetap memiliki lebar bit yang cukup besar untuk menjaga ketelitian, tetapi operand utama yang masuk ke multiplier tetap berada pada ukuran yang sesuai dengan karakteristik DSP FPGA.

Secara keseluruhan, akselerator CNN dirancang sebagai rangkaian streaming yang terdiri dari beberapa blok utama, yaitu pembentuk window berbasis *shift register*, unit MAC berbasis DSP, FSM pengendali, FIFO antarblok, serta mekanisme *valid-ready*. Kombinasi ini membuat model CNN dapat dijalankan pada FPGA dengan menyesuaikan antara kebutuhan komputasi, keterbatasan resource, dan kestabilan aliran data. Penggunaan resource yang tinggi, terutama pada DSP, masih dapat diterima selama rancangan memenuhi timing dan menghasilkan keluaran inferensi yang sesuai dengan model acuan.

3.2.5 Rancangan Sistem Kamera, DDR, dan VGA

3.3 Metode Implementasi

3.3.1 Implementasi Model CNN di Python

3.3.2 Implementasi Kuantisasi Fixed-Point

3.3.3 Implementasi CNN ke Verilog

3.3.4 Implementasi OV7670

3.3.5 Implementasi VGA

3.3.6 Implementasi DDR sebagai Frame Buffer

3.4 Metode Integrasi

3.4.1 Integrasi Model CNN ke FPGA

3.4.2 Integrasi OV7670 dengan DDR

3.4.3 Integrasi DDR dengan VGA

3.4.4 Integrasi OV7670, DDR, VGA, dan CNN

3.5 Metode Pengujian

3.5.1 Pengujian Akurasi Model di Python

3.5.2 Pengujian Perbedaan Hasil Python dan Verilog

3.5.3 Pengujian Penggunaan Kapasitas FPGA

3.5.4 Pengujian Sanitasi Kamera

3.5.5 Pengujian Sanitasi VGA

```

1 case (h_count[9:7])
2 // White
3 3'd0: begin vga_r <= 4'hF;vga_g <= 4'hF;vga_b <= 4'hF;end
4 // Yellow
5 3'd1: begin vga_r <= 4'hF;vga_g <= 4'hF;vga_b <= 4'h0; end
6 // Cyan
7 3'd2: begin vga_r <= 4'h0;vga_g <= 4'hF;vga_b <= 4'hF; end

```

```

8 // Green
9 3'd3: begin vga_r <= 4'h0;vga_g <= 4'hF;vga_b <= 4'h0; end
10 // Magenta
11 3'd4: begin vga_r <= 4'hF;vga_g <= 4'h0;vga_b <= 4'hF; end
12 // Red
13 3'd5: begin vga_r <= 4'hF;vga_g <= 4'h0;vga_b <= 4'h0; end
14 // Blue
15 3'd6: begin vga_r <= 4'h0;vga_g <= 4'h0;vga_b <= 4'hF; end
16 // Black
17 default: begin vga_r <= 4'h0;vga_g <= 4'h0;vga_b <= 4'h0;
    end
18 endcase

```

Kode Sanitasi VGA Verilog

3.5.6 Pengujian DDR

3.5.7 Pengujian Sistem Terintegrasi

3.6 Metode Evaluasi

3.6.1 Evaluasi Akurasi

3.6.2 Evaluasi Perbedaan Bit

3.6.3 Evaluasi Latensi Inferensi

3.6.4 Evaluasi Penggunaan Resource FPGA

3.6.5 Evaluasi Daya

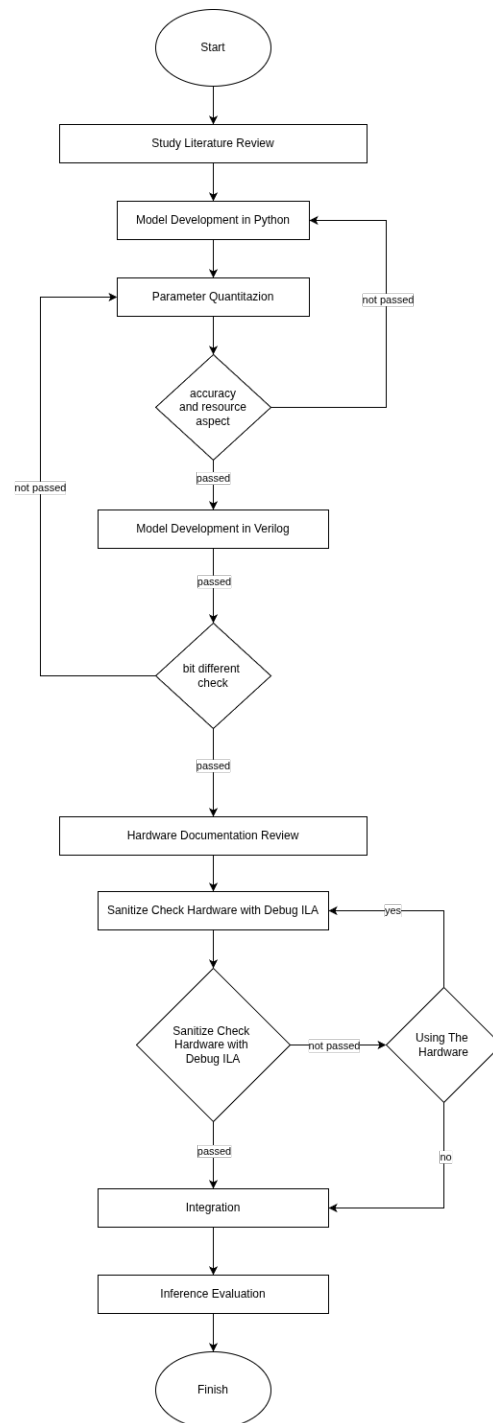
3.6.6 Evaluasi Tampilan Real Time

3.7 Tahapan Pelaksanaan

Tahapan pelaksanaan penelitian ini ditunjukkan pada Gambar 3.2. Penelitian diawali dengan studi literatur untuk memahami konsep implementasi CNN pada FPGA, teknik kuantisasi *fixed-point*, serta sistem pendukung seperti kamera, memori DDR, dan tampilan VGA. Setelah itu, model CNN dikembangkan terlebih dahulu menggunakan Python sebagai model acuan. Model yang telah diperoleh kemudian dikonversi ke representasi *fixed-point* 16-bit agar lebih sesuai dengan karakteristik

komputasi FPGA. Tahap kuantisasi ini divalidasi berdasarkan dua aspek, yaitu akurasi model dan kebutuhan *resource* FPGA, karena implementasi CNN pada FPGA perlu mempertimbangkan keseimbangan antara performa komputasi, penggunaan sumber daya, dan ketelitian numerik [11, 30].

Setelah model hasil kuantisasi dinilai memenuhi kebutuhan, arsitektur CNN diimplementasikan ke dalam Verilog. Hasil implementasi Verilog kemudian dibandingkan dengan hasil model Python melalui pengujian *bit-difference* untuk memastikan bahwa proses komputasi pada hardware tetap mengikuti model acuan. Tahap berikutnya adalah pengujian dasar hardware menggunakan *debug* ILA untuk memastikan blok kamera, FIFO, DDR, dan VGA dapat bekerja dengan benar sebelum dilakukan integrasi sistem. Setelah seluruh blok tervalidasi, sistem diintegrasikan menjadi alur lengkap mulai dari pengambilan citra, pemrosesan CNN, hingga evaluasi hasil inferensi.



Gambar 3.2: Diagram Alir Penelitian

3.8 Jadwal Kegiatan

Penelitian dilaksanakan selama enam bulan, yaitu dari Januari 2026 hingga Juni 2026 sebagaimana ditunjukkan pada Tabel 3.2. Tahap studi literatur dilakukan pada bulan pertama hingga bulan ketiga sebagai dasar dalam penyusunan dan pengembangan penelitian. Desain sistem dilaksanakan mulai bulan kedua hingga bulan keenam dengan mengacu pada hasil studi literatur serta penelitian terdahulu yang relevan. Pembuatan prototipe dilakukan secara paralel dengan proses desain sistem pada bulan keempat hingga bulan keenam dengan mempertimbangkan kapasitas perangkat keras dan waktu penelitian yang tersedia. Uji coba dan perbaikan sistem juga dilaksanakan pada periode yang sama untuk mengevaluasi kinerja prototipe dan melakukan penyempurnaan secara bertahap. Adapun penulisan skripsi dilakukan pada bulan keenam sebagai tahap akhir penelitian.

Tabel 3.2: Jadwal Penelitian.

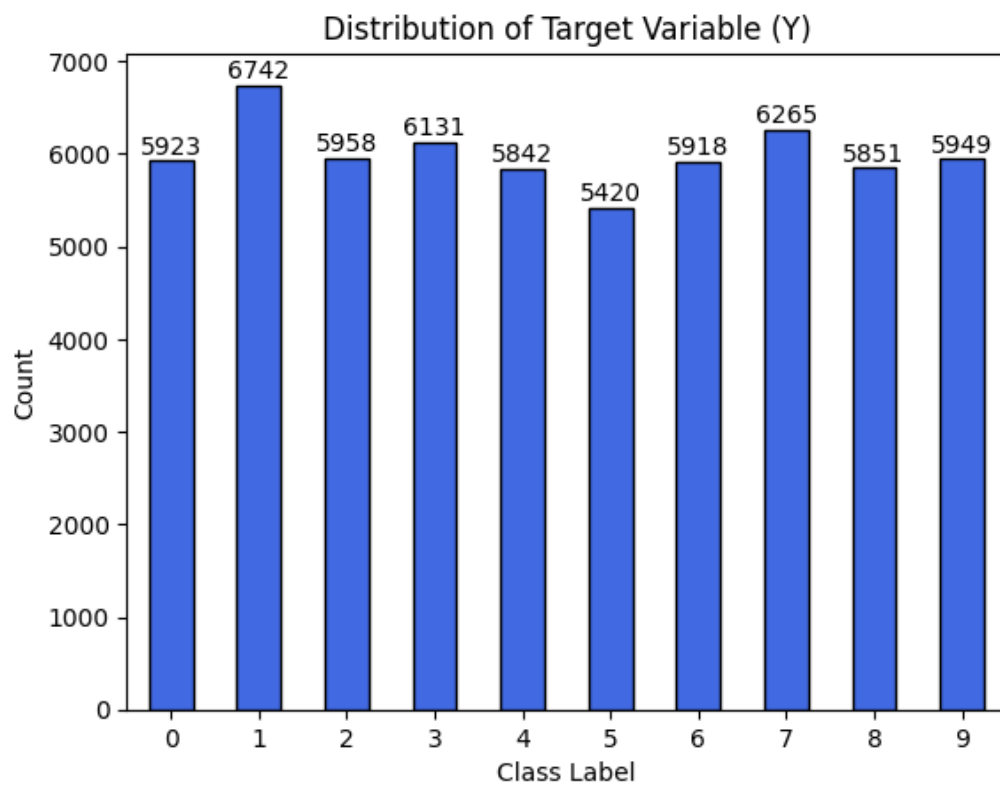
No	Keterangan	Bulan					
		1	2	3	4	5	6
1	Studi literatur						
2	Desain						
3	Pembuatan prototipe						
4	Uji coba dan perbaikan						
5	Penulisan skripsi						

BAB IV

HASIL DAN PEMBAHASAN

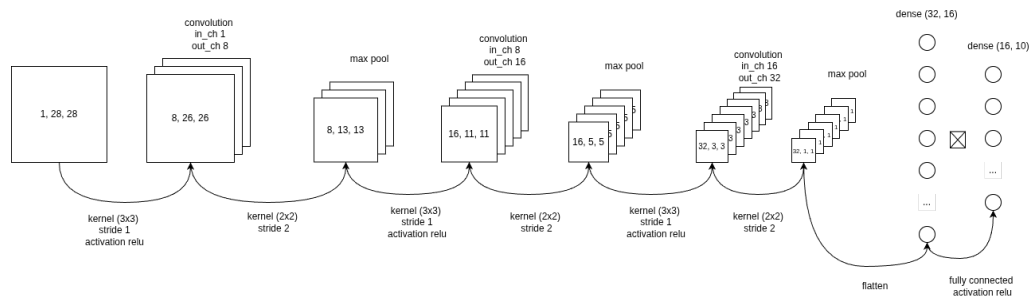
4.1 Hasil Implementasi Model CNN

4.1.1 Dataset yang Digunakan



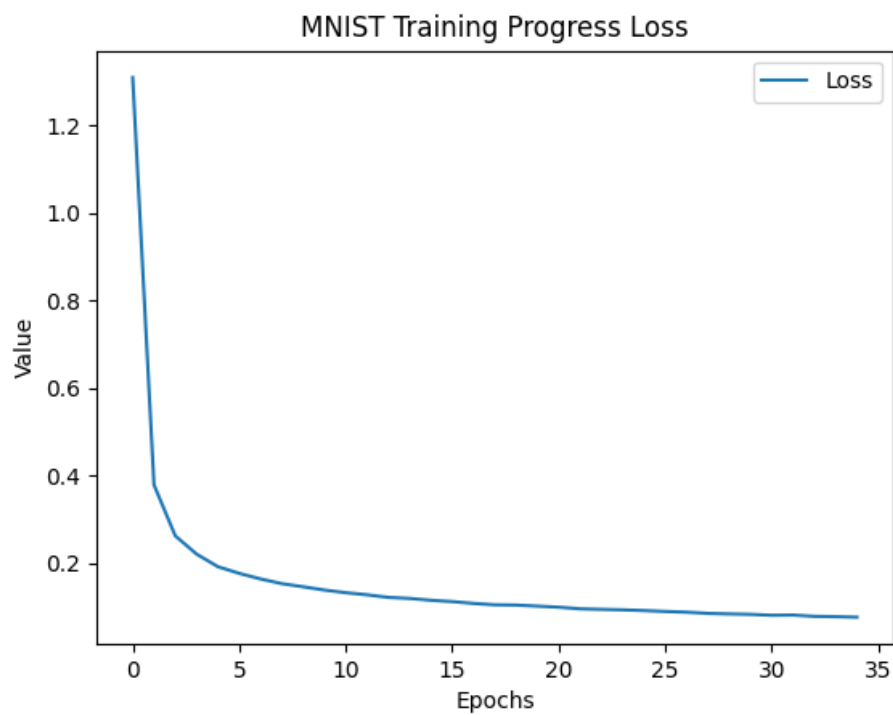
Gambar 4.1: Distribusi *Training Dataset* yang digunakan

4.1.2 Arsitektur Target Model CNN

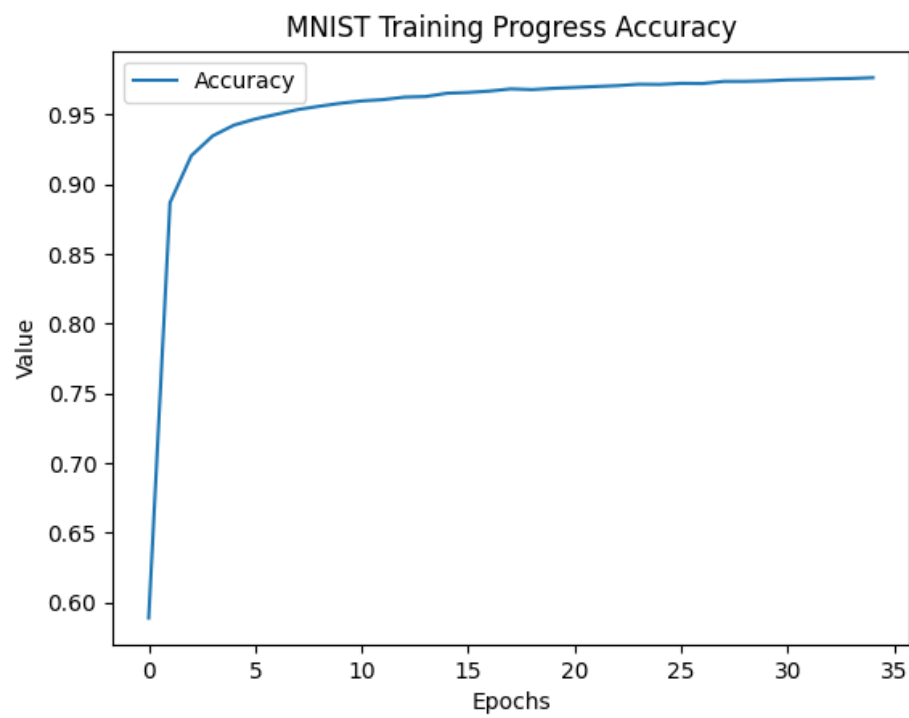


Gambar 4.2: Arsitektur Model CNN

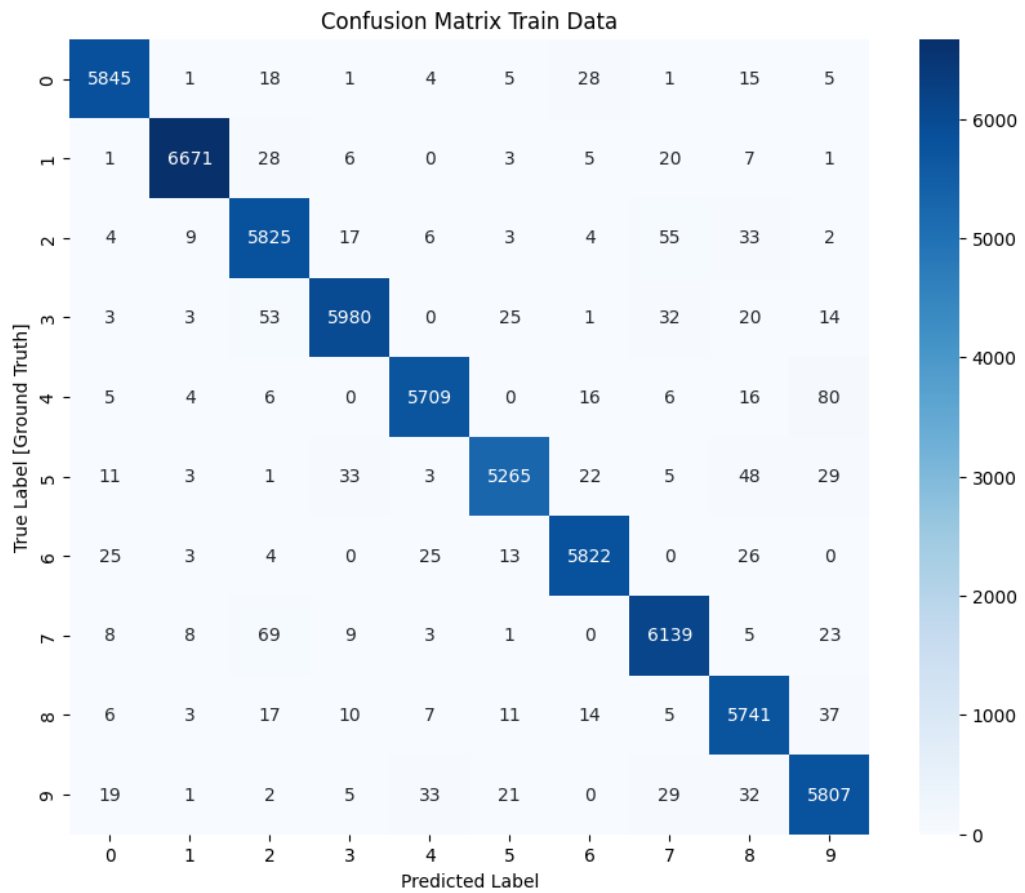
4.1.3 Hasil Pelatihan Model CNN



Gambar 4.3: *Loss training model*



Gambar 4.4: *Accuracy training model*

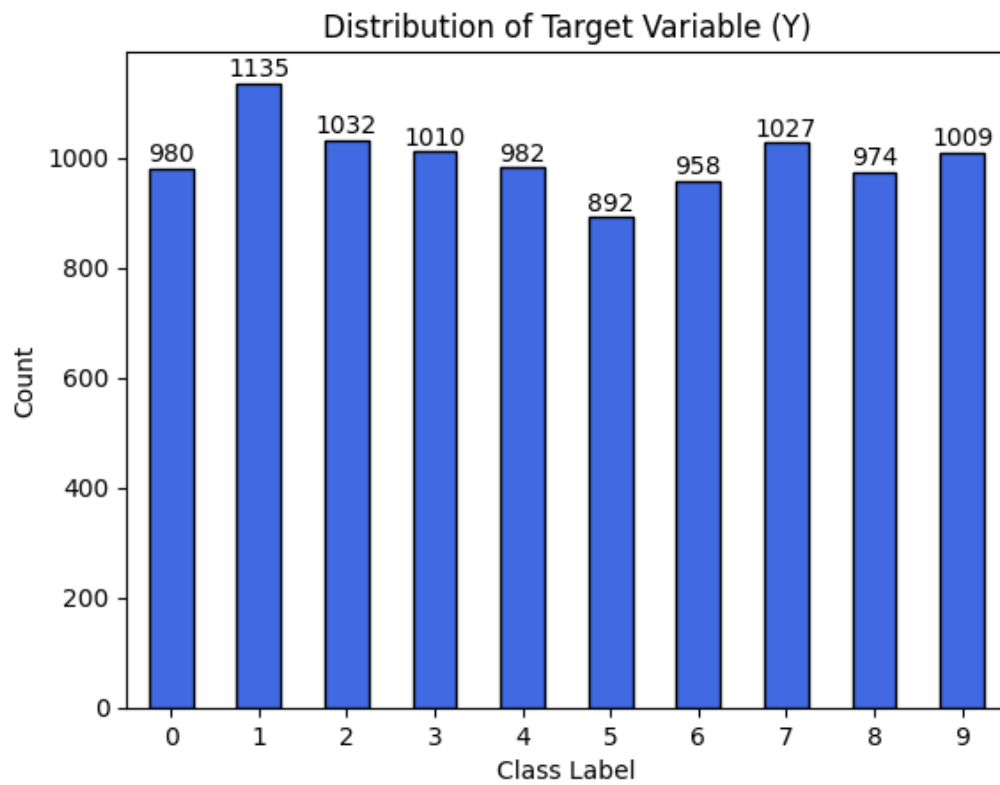


Gambar 4.5: *Confusion Matrix Training model*

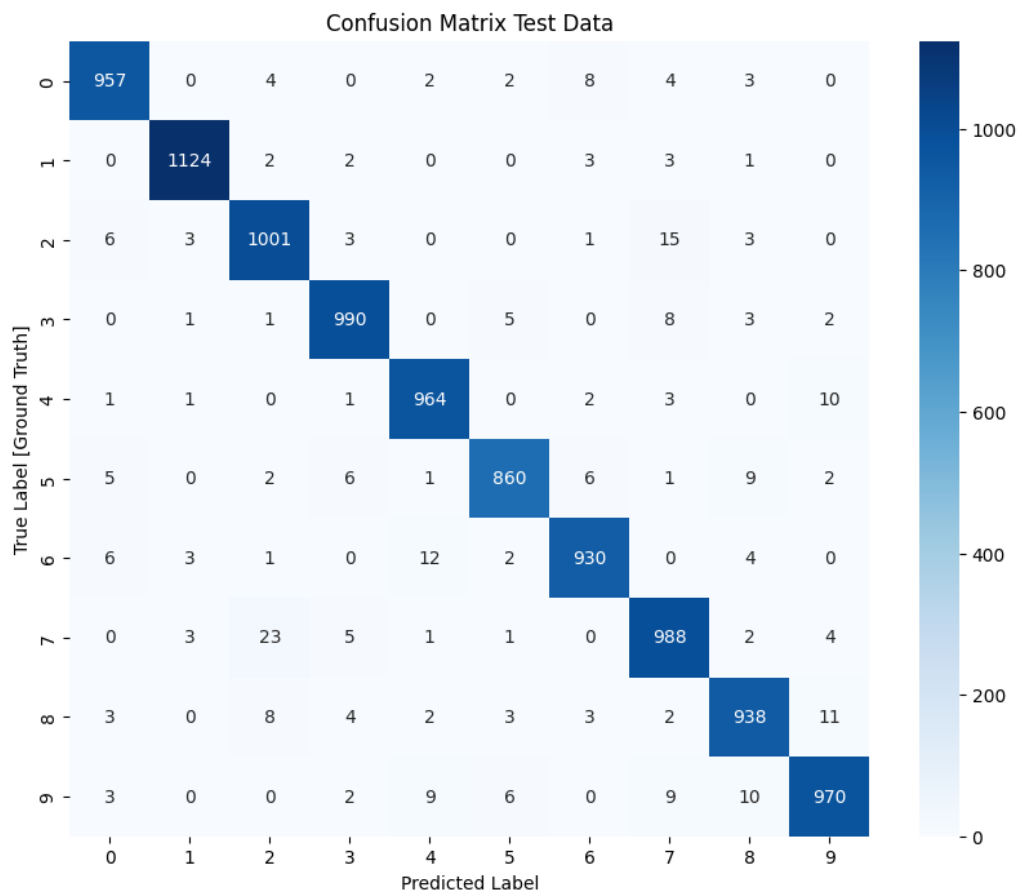
Tabel 4.1: Ukuran Parameter Pada Model Python

Parameter Name	Activation	Params	Parameter Size
conv2d	relu	80	992 B
pool	-	-	-
conv2d	relu	1168	9.47 KB
pool	-	-	-
conv2d	relu	4640	36.59 KB
pool	-	-	-
flatten	-	-	-
neural	relu	528	4.47 KB
neural	relu	170	1.67 KB
softmax	-	-	-
Total		6586	53,192 KB

4.1.4 Hasil Evaluasi Model CNN di Python



Gambar 4.6: Distribusi *Test Dataset* yang digunakan



Gambar 4.7: *Confusion Matrix testing model*

Tabel 4.2: Parameter Model pada Python

Parameter Name	Parameter Shape	Float Min	Float Max	Total Param
conv1_w	(8, 1, 3, 3)	-0.41649	0.22918	72
conv1_b	(8,)	-0.12903	0.20184	8
conv2_w	(16, 8, 3, 3)	-0.4300774	0.27402872	1152
conv2_b	(16,)	-0.14580758	0.16609082	16
conv3_w	(32, 16, 3, 3)	-0.27862	0.29362	4608
conv3_b	(32,)	-0.11603	0.10558	32
fc1_w	(16, 32)	-0.58693	0.56456	512
fc1_b	(16,)	-0.00994	0.01384	16
fc2_w	(10, 16)	-0.58495	0.81792	160
fc2_b	(10,)	-0.07052	0.13608	10

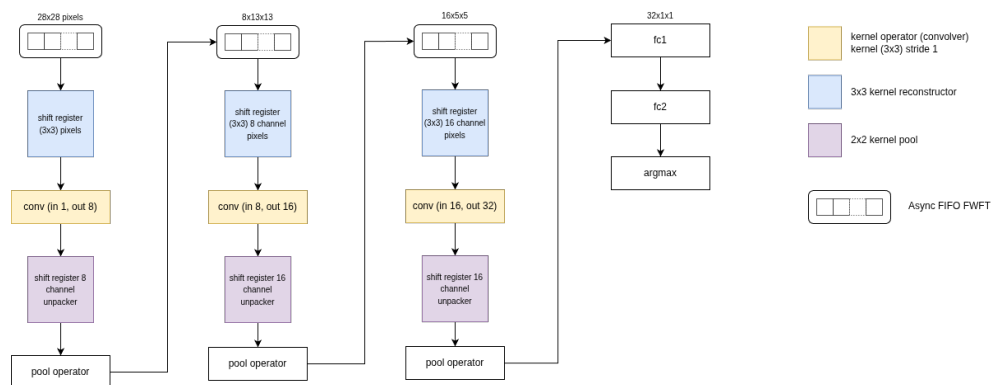
4.1.5 Kuantisasi Parameter Model CNN

Tabel 4.3: Hasil Kuantisasi Parameter

Parameter Name	Mem Row (Lines)	Low Sat.	High Sat.	Quant Min	Quant Max
conv1_w	72	0	0	-6824	3755
conv1_b	8	0	0	-2114	3307
conv2_w	1152	0	0	-7046	4490
conv2_b	16	0	0	-2389	2721
conv3_w	4608	0	0	-4565	4811
conv3_b	32	0	0	-1901	1730
fc1_w	512	0	0	-9616	9250
fc1_b	16	0	0	-163	227
fc2_w	160	0	0	-9584	13401
fc2_b	10	0	0	-1155	2230

4.2 Hasil Implementasi CNN Accelerator pada FPGA

4.2.1 Arsitektur CNN Accelerator pada FPGA



Gambar 4.8: Desain CNN Accelerator pada FPGA

4.2.2 Hasil Penggunaan Resource CNN Accelerator

Tabel 4.4: Hasil *CNN Implement Running*

Komponen	Resource Terpakai	Persentase %
LUT	13.742	22.11
Flip-Flop	27.135	21.4
Block RAM	18	13.33
DSP Slice	232 DSP48E1	97.91

4.2.3 Hasil Timing CNN Accelerator

Tabel 4.5: Hasil Routing CNN Accelerator Design

Conflict Nets	Unrouted Nets	Partially Routed Nets	Fully Routed Nets
0	0	0	44973

Tabel 4.6: Hasil *CNN Accelerator Delayed Design*

WNS	TNS	WHS	WBSS	TPWS
0.121	0.00	0.010	0.00	3.75

4.2.4 Hasil Latensi dan Throughput CNN Accelerator

Tabel 4.7: Hasil Latensi CNN Accelerator

Tahap	Jumlah Siklus	Latensi pada 100 MHz
Block 1 Output	98	0,98 μs
Block 2 Output	545	5,45 μs
Frame Done	788	7,88 μs
Block 3 Output	2457	24,57 μs
Dense Output	3802	38,02 μs

$$Throughput = \frac{1}{38,02 \times 10^{-6}} = 26302 \text{ inferensi/s} \quad (4.1)$$

4.3 Hasil Pengujian Modul Perangkat Keras

4.3.1 Hasil Uji Sanitasi OV7670

4.3.2 Hasil Uji Sanitasi VGA Pmod

4.3.3 Hasil Uji Akses DDR3 SDRAM

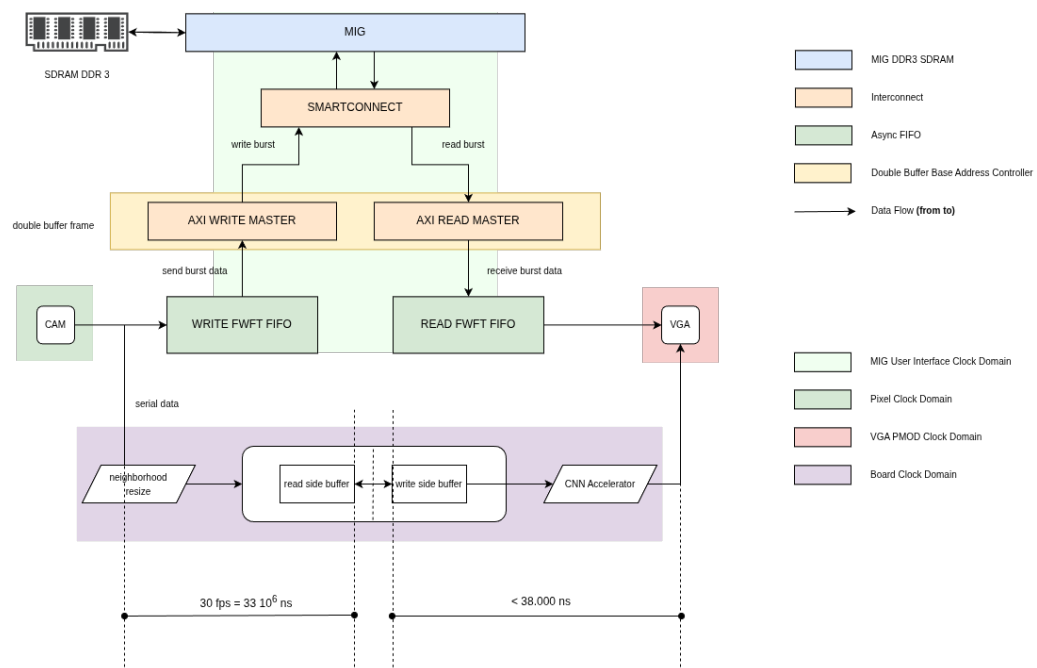
4.4 Hasil Integrasi Sistem

4.4.1 Hasil Integrasi OV7670, DDR, dan VGA

Tabel 4.8: Hasil *Streaming Flow Implement Running*

Komponen	Resource Terpakai	Persentase %
LUT	6.969	11
Flip-Flop	6.357	5.01
Block RAM	0.5	0.37
DSP Slice	0	0.0

4.4.2 Hasil Integrasi CNN Accelerator dengan OV7670, DDR, dan VGA



Gambar 4.9: Desain Akhir Integrasi

4.4.3 Hasil Penggunaan Resource Sistem Terintegrasi

Tabel 4.9: Detail Hasil *Implement Running*

Komponen	Resource Terpakai	Persentase %
LUT	20.755	32.74
Flip-Flop	34.502	27.21
Block RAM	19.5	14.44
DSP Slice	235 DSP48E1	97.92

4.4.4 Hasil Timing Sistem Terintegrasi

Tabel 4.10: Hasil Routing Design

Conflict Nets	Unrouted Nets	Partially Routed Nets	Fully Routed Nets
0	0	0	63141

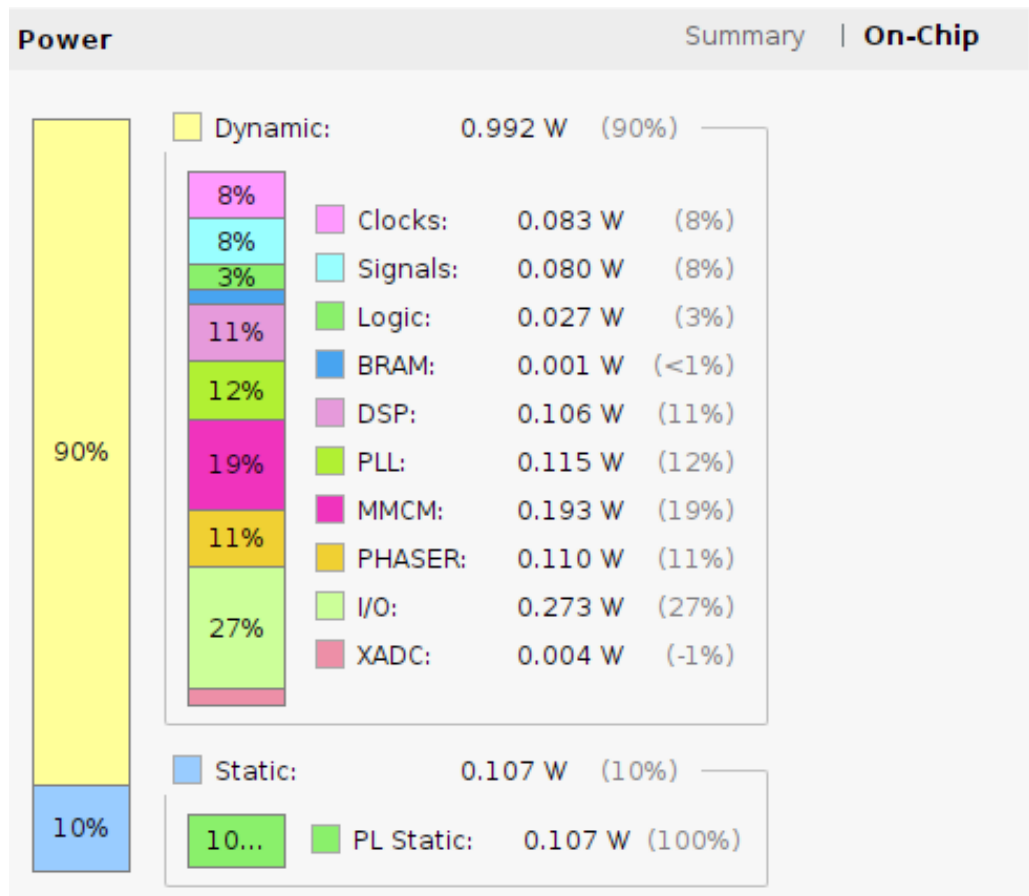
Tabel 4.11: Hasil *Delayed Design*

WNS	TNS	WHS	WBSS	TPWS
0.177	0.00	0.012	0.00	11.305

4.4.5 Hasil Estimasi Daya Sistem Terintegrasi

Tabel 4.12: Penggunaan Daya dan Temperature

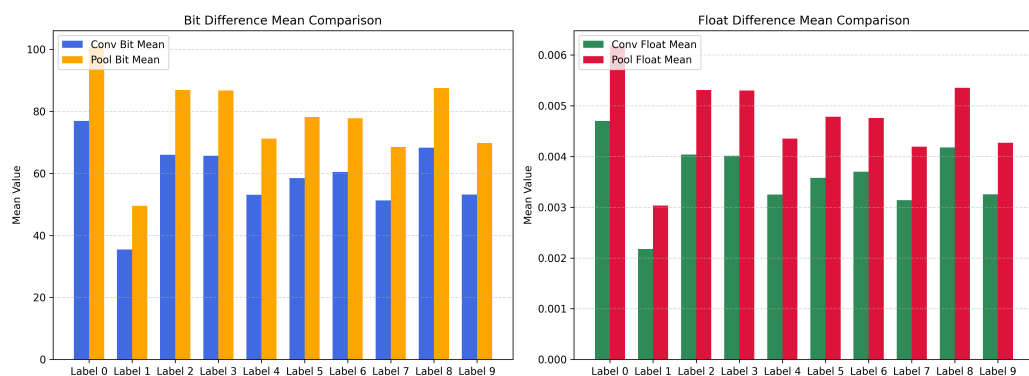
Total On Chip	Junction Temperature	Thermal Margin	Effective JA
1.099 W	30.0 C	55.0 C (11.9 W)	4.6 C/W



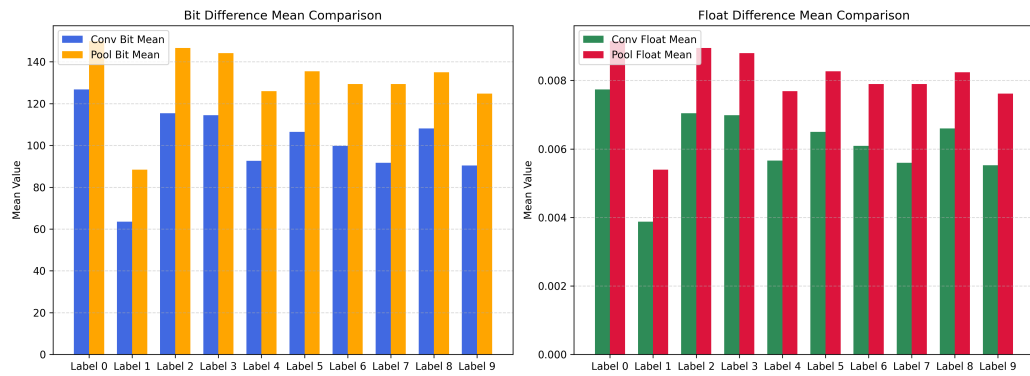
Gambar 4.10: Detail Hasil Penggunaan Daya

4.5 Hasil Evaluasi dan Pembahasan

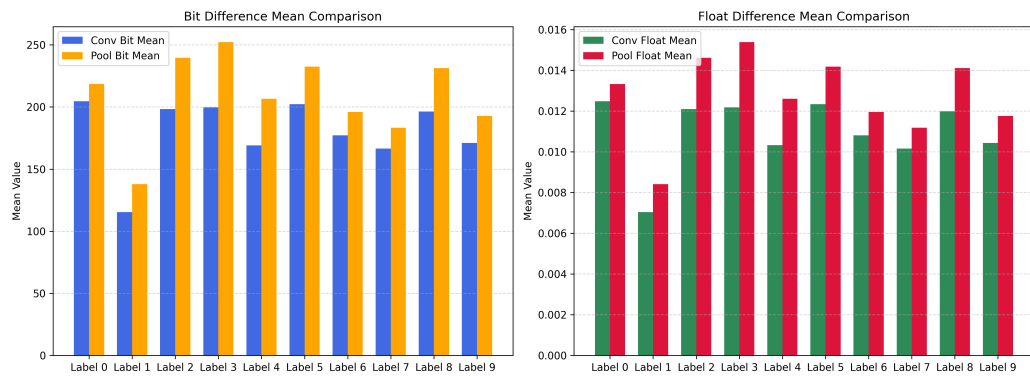
4.5.1 Evaluasi Perbedaan Hasil Python dan Verilog



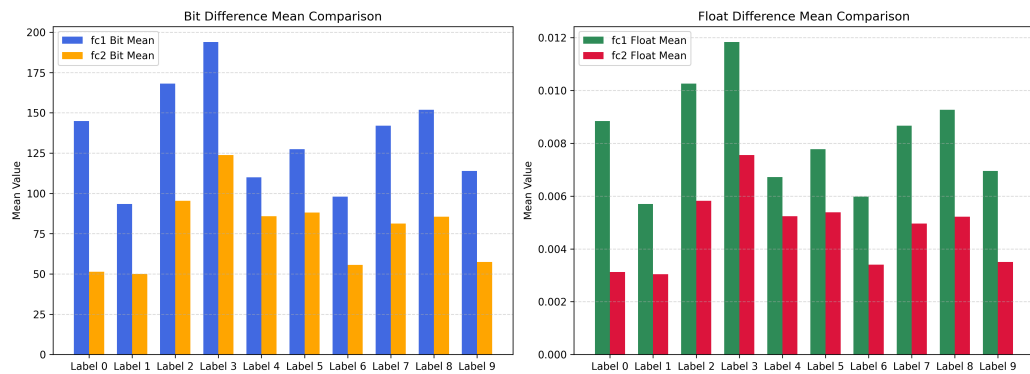
Gambar 4.11: Perbandingan *Bit* dan *Float value* pada *Conv 1* dan *Pool 1*



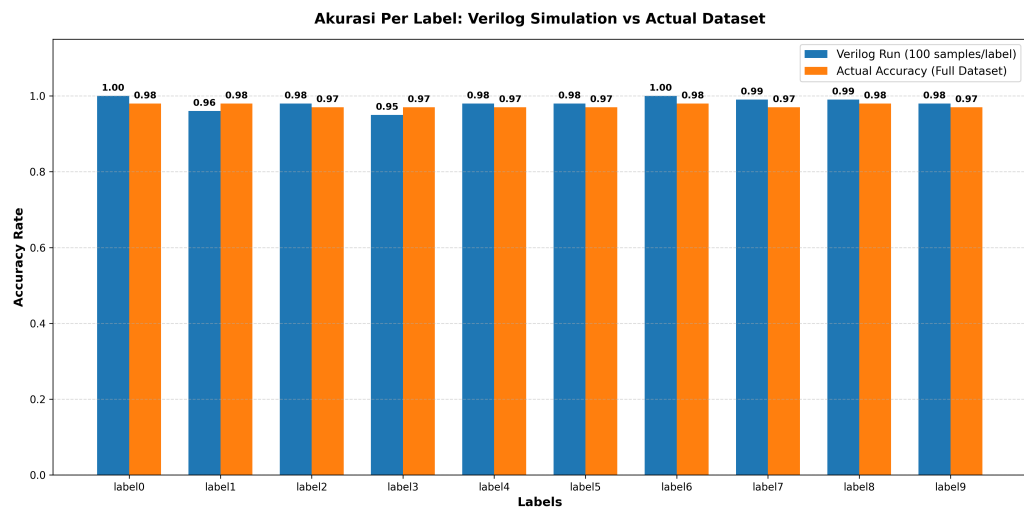
Gambar 4.12: Perbandingan *Bit* dan *Float value* pada *Conv 2* dan *Pool 2*



Gambar 4.13: Perbandingan *Bit* dan *Float value* pada *Conv 3* dan *Pool 3*



Gambar 4.14: Perbandingan *Bit* dan *Float value* pada *Dense 1* dan *Dense 2*



Gambar 4.15: Perbedaan Akurasi

4.5.2 Evaluasi Hasil Inferensi Model pada FPGA

4.5.3 Pembahasan Resource, Latensi, dan Daya

4.5.4 Pembahasan Ketercapaian Sistem terhadap Tujuan Penelitian

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan beberapa hal sebagai berikut.

1. Model CNN berhasil diimplementasikan pada board FPGA menggunakan representasi fixed-point 16-bit. Proses kuantisasi dilakukan terhadap parameter bobot dan bias model sehingga dapat digunakan pada modul Verilog tanpa menggunakan operasi floating-point.
2. Hasil implementasi CNN accelerator menunjukkan bahwa sistem mampu menghasilkan keluaran inferensi dalam 3802 siklus clock. Pada frekuensi kerja 100 MHz, nilai tersebut setara dengan latensi sebesar 38,02 μ s.
3. Hasil penggunaan resource menunjukkan bahwa CNN accelerator menggunakan sebagian besar DSP pada FPGA. Pada implementasi sistem penuh, penggunaan DSP mencapai 235 DSP48E1 dari total 240 DSP yang tersedia, sehingga DSP menjadi resource utama yang membatasi pengembangan model.
4. Sistem streaming citra berbasis OV7670, DDR3 SDRAM, dan VGA berhasil dirancang dan diintegrasikan sebagai bagian dari sistem pendukung akuisisi dan tampilan citra pada FPGA.
5. Hasil implementasi sistem penuh menunjukkan bahwa desain dapat melalui proses routing tanpa unrouted net dan memenuhi timing dengan nilai WNS positif. Estimasi daya total on-chip yang diperoleh adalah sebesar 1,099 W.

5.2 Saran

Berdasarkan penelitian yang telah dilakukan, terdapat beberapa saran untuk pengembangan selanjutnya sebagai berikut.

1. Optimasi penggunaan DSP perlu dilakukan karena implementasi CNN accelerator menggunakan resource DSP dalam jumlah sangat besar. Optimasi dapat

dilakukan melalui penggunaan time multiplexing, pengurangan jumlah channel, atau perancangan ulang arsitektur CNN yang lebih ringan.

2. Pengujian sistem dapat dikembangkan menggunakan data citra yang diperoleh langsung dari kamera OV7670 agar sistem klasifikasi lebih merepresentasikan kondisi penggunaan secara real-time.
3. Pengukuran daya dapat dilakukan menggunakan alat ukur eksternal agar hasil konsumsi daya tidak hanya bergantung pada estimasi dari Vivado.

DAFTAR PUSTAKA

- [1] O. Elharrouss, Y. Akbari, N. Almaadeed, and S. A. Al-Maadeed, “Backbones-review: Feature extraction networks for deep learning and deep reinforcement learning approaches,” *ArXiv*, vol. abs/2206.08016, 2022. [Online]. Available: <https://api.semanticscholar.org/CorpusID:249712263>
- [2] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “Imagenet classification with deep convolutional neural networks,” in *Advances in Neural Information Processing Systems*, F. Pereira, C. Burges, L. Bottou, and K. Weinberger, Eds., vol. 25. Curran Associates, Inc., 2012. [Online]. Available: <https://neurips.cc>
- [3] X. Liao, Y. Li, S. Zhang, X. Wei, and J. Hu, “Predicting gpu training energy consumption in data centers using task metadata via symbolic regression,” *Energies*, vol. 19, no. 2, 2026. [Online]. Available: <https://www.mdpi.com/1996-1073/19/2/448>
- [4] S. Stirenko, Y. Rosenberg, Y. Gordienko, O. Alienin, O. Rokovyi, and N. Alienina, “Batch size influence on performance of graphic and tensor processing units during training and inference phases,” *arXiv preprint arXiv:1812.11731*, pp. 1–9, 2018. [Online]. Available: <https://arxiv.org/abs/1812.11731>
- [5] S. Grigorescu, B. Trasnea, T. Cocias, and G. Macesanu, “A survey of deep learning techniques for autonomous driving,” *Journal of Field Robotics*, vol. 37, no. 3, pp. 362–386, 2020.
- [6] K. Guo, S. Zeng, J. Yu, Y. Wang, and H. Yang, “A survey of fpga-based neural network inference accelerators,” *ACM Transactions on Reconfigurable Technology and Systems*, vol. 12, no. 1, 2019.
- [7] K. Abdelouahab, M. Pelcat, J. Serot, and F. Berry, “Accelerating cnn inference on fpgas: A survey,” *arXiv preprint arXiv:1806.01683*, 2018.
- [8] S. Che, M. Boyer, J. Meng, D. Tarjan, J. W. Sheaffer, S.-H. Lee, and K. Skadron, “A performance study of general-purpose applications on graphics processors,” *Journal of Parallel and Distributed Computing*, vol. 69, no. 10, pp. 1370–1380, 2009.

- [9] M. Qasaimeh, K. Denolf, J. Lo, K. Vissers, M. Mattavelli, and J. Silas, "Comparing energy efficiency of cpu, gpu and fpga implementations for vision kernels," in *2019 IEEE International Conference on Embedded Software and Systems (ICCESS)*. IEEE, 2019, pp. 1–8.
- [10] Xilinx Inc., "Lowering power at 28 nm with xilinx 7 series fpgas," White Paper: WP389 (v1.2), 2015.
- [11] R. Solovyev, A. Kustov, D. Telpukhov, V. Rukhlov, and A. Kalinin, "Fixed-point convolutional neural network for real-time video processing in FPGA," in *2019 IEEE Conference of Russian Young Researchers in Electrical and Electronic Engineering (EIconRus)*. IEEE, 2019, pp. 1605–1611. [Online]. Available: <https://doi.org/10.1109/EIconRus.2019.8656778>
- [12] *OV7670/OV7171 CMOS VGA CameraChip Sensor Datasheet*, OmniVision Technologies, 2006, accessed: 2026-06-15. [Online]. Available: https://web.mit.edu/6.111/www/f2016/tools/OV7670_2006.pdf
- [13] J. Qiu, J. Wang, S. Yao, K. Guo, B. Li, E. Zhou, J. Yu, T. Tang, N. Xu, S. Song, Y. Wang, and H. Yang, "Going deeper with embedded FPGA platform for convolutional neural network," in *Proceedings of the 2016 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, ser. FPGA '16. Association for Computing Machinery, 2016, pp. 26–35. [Online]. Available: <https://doi.org/10.1145/2847263.2847265>
- [14] A. Dua, Y. Li, and F. Ren, "Systolic-CNN: An OpenCL-defined scalable run-time-flexible FPGA accelerator architecture for accelerating convolutional neural network inference in cloud/edge computing," 2020. [Online]. Available: <https://arxiv.org/abs/2012.03177>
- [15] S. Navaneethan, C. Thanusha, M. A. Varshini, and A. Anil, "A hardware efficient real-time video processing on FPGA with OV7670 camera interface and VGA," in *2022 6th International Conference on Electronics, Communication and Aerospace Technology (ICECA)*. IEEE, 2022, pp. 348–352. [Online]. Available: <https://doi.org/10.1109/ICECA55336.2022.10009129>
- [16] L. Sun, E. Sun, and Y. Yu, "Real time controllable multi channel HD digital video processing system based on FPGA and MicroBlaze," in *Proceedings of*

- the 2021 7th International Conference on Computing and Artificial Intelligence*, ser. ICCAI '21. Association for Computing Machinery, 2021, pp. 183–190. [Online]. Available: <https://doi.org/10.1145/3467707.3467734>
- [17] A. Cosoroaba, “Achieving high performance DDR3 data rates,” Xilinx, White Paper WP383, Aug. 2013. [Online]. Available: <https://docs.amd.com/api/khub/documents/SKHv3pFeACeh9A7QJryuMg/content>
- [18] A. Jacobo, “FPGA_OV7670_Camera_Interface: Real-time streaming of OV7670 camera via VGA,” GitHub repository, 2021, accessed: 2026-06-15. [Online]. Available: https://github.com/AngeloJacob/FPGA_OV7670_Camera_Interface
- [19] Arm Limited, *AMBA AXI and ACE Protocol Specification*, Arm Limited, 2020, non-Confidential. [Online]. Available: <https://developer.arm.com/documentation/ihi0022/latest>
- [20] —, *Introduction to AMBA AXI*, Arm Limited, 2020. [Online]. Available: <https://developer.arm.com/documentation/102202/latest>
- [21] Xilinx, *7 Series DSP48E1 Slice User Guide*, Xilinx, 2018. [Online]. Available: https://docs.amd.com/v/u/en-US/ug479_7Series_DSP48E1
- [22] F. Yan, F. Wang, C. Liu, Y. Wang, and J. Zhou, “Design of convolutional neural network processor based on fpga resource multiplexing architecture,” *Sensors*, vol. 22, no. 16, p. 5967, 2022. [Online]. Available: <https://www.mdpi.com/1424-8220/22/16/5967>
- [23] C. Zhang, P. Li, G. Sun, Y. Guan, B. Xiao, and J. Cong, “Optimizing FPGA-based accelerator design for deep convolutional neural networks,” in *Proceedings of the 2015 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, ser. FPGA '15. Association for Computing Machinery, 2015, pp. 161–170. [Online]. Available: <https://doi.org/10.1145/2684746.2689060>
- [24] Y. Umuroglu, N. J. Fraser, G. Gambardella, M. Blott, P. Leong, M. Jahre, and K. Vissers, “FINN: A framework for fast, scalable binarized neural network inference,” in *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, ser. FPGA '17.

- Association for Computing Machinery, 2017, pp. 65–74. [Online]. Available: <https://doi.org/10.1145/3020078.3021744>
- [25] AMD, “68595 - DSP slice - using time division multiplexing or overclocking the DSP slices in Xilinx devices can greatly increase throughput and efficiency,” https://adaptivesupport.amd.com/s/article/68595?language=en_US, Feb. 2023, diakses pada: 16 Juni 2026.
- [26] C. E. Cummings and P. Alfke, “Simulation and synthesis techniques for asynchronous fifo design,” in *SNUG San Jose*. Sunburst Design, 2002. [Online]. Available: https://www.sunburst-design.com/papers/CummingsSNUG2002SJ_FIFO1.pdf
- [27] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016. [Online]. Available: <https://www.deeplearningbook.org/>
- [28] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [29] V. Nair and G. E. Hinton, “Rectified linear units improve restricted boltzmann machines,” in *Proceedings of the 27th International Conference on Machine Learning*, 2010, pp. 807–814.
- [30] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 2704–2713. [Online]. Available: https://openaccess.thecvf.com/content_cvpr_2018/html/Jacob_Quantization_and_Training_CVPR_2018_paper.html
- [31] S. Gupta, A. Agrawal, K. Gopalakrishnan, and P. Narayanan, “Deep learning with limited numerical precision,” in *Proceedings of the 32nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 37. PMLR, 2015, pp. 1737–1746. [Online]. Available: <https://proceedings.mlr.press/v37/gupta15.html>
- [32] D. Lin, S. Talathi, and S. Annapureddy, “Fixed point quantization of deep convolutional networks,” in *Proceedings of the 33rd International*

- Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 48. PMLR, 2016, pp. 2849–2858. [Online]. Available: <https://proceedings.mlr.press/v48/linb16.html>
- [33] R. Goyal, J. Vanschoren, V. van Acht, and S. Nijssen, “Fixed-point quantization of convolutional neural networks for quantized inference on embedded platforms,” *arXiv preprint arXiv:2102.02147*, 2021. [Online]. Available: <https://arxiv.org/abs/2102.02147>
- [34] R. Krishnamoorthi, “Quantizing deep convolutional networks for efficient inference: A whitepaper,” *arXiv preprint arXiv:1806.08342*, 2018. [Online]. Available: <https://arxiv.org/abs/1806.08342>
- [35] Arm Ltd., *AMBA AXI and ACE Protocol Specification*, Arm Ltd., Dec. 2017. [Online]. Available: https://developer.arm.com/-/media/Arm%20Developer%20Community/PDF/IHI0022H_amba_axi_protocol_spec.pdf
- [36] Xilinx, *7 Series FPGAs Memory Interface Solutions User Guide*, Xilinx, Apr. 2012. [Online]. Available: https://www.xilinx.com/support/documents/ip_documentation/mig_7series/v4_1/ug586_7Series_MIS.pdf
- [37] Digilent, *VGA Display Controller*, Digilent Inc., 2024. [Online]. Available: <https://digilent.com/reference/learn/programmable-logic/tutorials/vga-display-controller/start>
- [38] K. Guo, S. Zeng, J. Yu, Y. Wang, and H. Yang, “A survey of fpga-based neural network inference accelerators,” *ACM Transactions on Reconfigurable Technology and Systems*, vol. 12, no. 1, pp. 1–26, 2019. [Online]. Available: <https://dl.acm.org/doi/10.1145/3289185>
- [39] H.-J. Kang, “AoCStream: All-on-chip CNN accelerator with stream-based line-buffer architecture,” in *Proceedings of the 2023 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, ser. FPGA ’23. Association for Computing Machinery, 2023, p. 48. [Online]. Available: <https://doi.org/10.1145/3543622.3573141>
- [40] J. F. Wakerly, *Digital Design: Principles and Practices*, 4th ed. Pearson Prentice Hall, 2005.

- [41] S. L. Harris and D. Harris, *Digital Design and Computer Architecture*, 2nd ed. Morgan Kaufmann, 2021.
- [42] J. H. Kim, B. Grady, R. Lian, J. Brothers, and J. H. Anderson, “FPGA-based CNN inference accelerator synthesized from multi-threaded C software,” in *2017 30th IEEE International System-on-Chip Conference (SOCC)*. IEEE, 2017, pp. 268–273. [Online]. Available: <https://doi.org/10.1109/SOCC.2017.8226056>

LAMPIRAN

L.1 Lampiran Hasil *Running Synthetis CNN Accelerator*